

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT VĨNH LONG

KHÓA LUẬN TỐT NGHIỆP
NGÀNH CÔNG NGHỆ THÔNG TIN

**ĐỀ TÀI: XÂY DỰNG HỆ THỐNG GIÁM SÁT
VÀ QUẢN LÝ BÃI ĐỖ XE THÔNG MINH
ỨNG DỤNG CÔNG NGHỆ AIOT
VÀ TRÍ TUỆ NHÂN TẠO**

CBHD: PGS.TS Phan Anh Cang

Sinh viên thực hiện: Trần Văn

Còn

Mã số sinh viên: 22004015

Vĩnh Long, năm 2026



BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT VĨNH LONG

KHOA CÔNG NGHỆ THÔNG TIN

KHÓA LUẬN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ THÔNG TIN

**ĐỀ TÀI: XÂY DỰNG HỆ THỐNG GIÁM SÁT
VÀ QUẢN LÝ BÃI ĐỖ XE THÔNG MINH
ỨNG DỤNG CÔNG NGHỆ AIOT
VÀ TRÍ TUỆ NHÂN TẠO**

CBHD: PGS.TS Phan Anh Cang

Sinh viên thực hiện: Trần Văn

Còn

Mã số sinh viên: 22004015

Vĩnh Long, năm 2026

PHIẾU GIAO KHÓA LUẬN TỐT NGHIỆP

Tên đề tài: ***Xây dựng hệ thống giám sát và quản lý bãi đỗ xe thông minh ứng dụng công nghệ AIoT và trí tuệ nhân tạo.***

- Nghiên cứu công nghệ định vị Ultra-Wideband (UWB), mô hình Anchor–Tag và thuật toán Trilateration trong bài toán xác định vị trí phương tiện ở không gian trong nhà; trên cơ sở đó thiết kế và lập trình firmware cho module UWB EWT550 kết hợp vi điều khiển ESP32-C3 (đảm nhận hai vai trò Anchor và Tag) nhằm thu thập và truyền dữ liệu khoảng cách theo thời gian thực qua UART và MQTT.
- Xây dựng IoT Server đảm nhiệm việc xử lý dữ liệu cảm biến, tính toán tọa độ phương tiện, lọc nhiễu kết quả định vị, lưu trữ lịch sử di chuyển và phân phối trạng thái theo thời gian thực.
- Xây dựng ứng dụng di động trên nền tảng React Native/Expo cho phép hiển thị bản đồ bãi đỗ xe cùng vị trí và trạng thái phương tiện được cập nhật theo thời gian thực.
- Nghiên cứu và triển khai chatbot hỏi đáp tiếng Việt ứng dụng kỹ thuật Retrieval-Augmented Generation (RAG), kết hợp truy xuất ngữ nghĩa và truy xuất từ khóa nhằm hỗ trợ tra cứu thông tin về hệ thống bãi đỗ xe.
- Tiến hành kiểm thử chức năng hệ thống và đánh giá độ chính xác của kết quả định vị trong điều kiện thực nghiệm.

Phương pháp đánh giá: ☐ Báo cáo trước hội đồng ☐ Chấm thuyết minh

Ngày giao khóa luận: ngày ... tháng ... năm 2026

Ngày hoàn thành khóa luận: ngày ... tháng ... năm 2026

Số lượng sinh viên thực hiện khóa luận: 1

Họ và tên sinh viên: Trần Văn Còn..... MSSV: 22004015

Vĩnh Long, ngày tháng năm 2026

Trưởng Khoa/Bộ môn

Người hướng dẫn

Phan Anh Cang

PGS.TS Phan Anh Cang

NHẬN XÉT & ĐÁNH GIÁ CỦA GV HƯỚNG DẪN

Ý thức thực hiện:

.....

.....

Nội dung thực hiện:

.....

.....

Hình thức trình bày:

.....

.....

Tổng hợp kết quả:

☐ Tổ chức báo cáo trước hội đồng

☐ Tổ chức chấm thuyết minh

Vĩnh Long, ngày tháng năm

Người hướng dẫn

PGS.TS Phan Anh Cang

LỜI CẢM ƠN

Khóa luận tốt nghiệp với đề tài "Xây dựng hệ thống giám sát và quản lý bãi đỗ xe thông minh ứng dụng công nghệ AIoT và trí tuệ nhân tạo" là kết quả của quá trình học tập và nghiên cứu của em tại Khoa Công nghệ Thông tin — Trường Đại học Sư phạm Kỹ thuật Vĩnh Long. Để hoàn thành khóa luận này, em đã nhận được sự hướng dẫn và hỗ trợ tận tình từ quý thầy cô.

Em xin gửi lời cảm ơn sâu sắc đến PGS.TS. Phan Anh Cang — giảng viên trực tiếp hướng dẫn em thực hiện đề tài. Thầy đã định hướng nghiên cứu và đưa ra những nhận xét chuyên môn quan trọng trong từng giai đoạn, từ thiết kế phần cứng định vị UWB, xây dựng IoT Server xử lý dữ liệu đến phát triển hệ thống chatbot hỏi đáp. Sự góp ý của thầy đã giúp em khắc phục nhiều hạn chế và hoàn thiện đề tài.

Em cũng xin cảm ơn quý thầy cô trong Khoa Công nghệ Thông tin đã trang bị cho em nền tảng kiến thức chuyên môn trong suốt quá trình học tập, tạo điều kiện để em tiếp cận và thực hiện một đề tài có tính ứng dụng cao.

Em xin gửi lời cảm ơn đến gia đình và bạn bè đã luôn ủng hộ, động viên em trong thời gian thực hiện khóa luận.

Do kiến thức và kinh nghiệm thực tiễn còn hạn chế, khóa luận không tránh khỏi những thiếu sót. Em rất mong nhận được ý kiến đóng góp của quý thầy cô để đề tài được hoàn thiện hơn.

Em xin chân thành cảm ơn!

MỤC LỤC

Mục lục tự động (nhấn F9 trong Word để cập nhật)

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Nội dung đầy đủ
1	AIoT	Artificial Intelligence of Things
2	UWB	Ultra-Wideband
3	TWR	Two-Way Ranging
4	MQTT	Message Queuing Telemetry Transport
5	TLS	Transport Layer Security
6	RAG	Retrieval-Augmented Generation
7	BM25	Best Match 25
8	RRF	Reciprocal Rank Fusion
9	SSE	Server-Sent Events
10	LLM	Large Language Model
11	NLP	Natural Language Processing
12	GPS	Global Positioning System
13	MCP	Model Context Protocol
14	TTL	Time To Live
15	API	Application Programming Interface
16	ORM	Object-Relational Mapping
17	GIN	Generalized Inverted Index
18	RSSI	Received Signal Strength Indicator
19	NLOS	Non-Line-of-Sight
20	LOS	Line-of-Sight

DANH MỤC HÌNH ẢNH

Danh mục hình tự động (nhấn F9 trong Word để cập nhật)

DANH MỤC BẢNG

Danh mục bảng tự động (nhấn F9 trong Word để cập nhật)

LỜI NÓI ĐẦU

Trong những năm gần đây, sự kết hợp giữa Internet vạn vật (IoT) và trí tuệ nhân tạo (AIoT) đã mở ra hướng tiếp cận mới cho bài toán quản lý không gian đô thị. Bãi đỗ xe trong nhà là một trong những môi trường điển hình thụ hưởng xu hướng này, song cũng đặt ra thách thức riêng: tín hiệu định vị vệ tinh (GNSS) bị suy giảm, khiến việc xác định vị trí phương tiện theo thời gian thực trở thành bài toán khó.

Các hướng định vị trong nhà phổ biến đều bộc lộ hạn chế: GNSS gần như mất tác dụng do bị che chắn bởi kết cấu bê tông; Wi-Fi RSSI và Bluetooth Low Energy (BLE) cho chi phí thấp nhưng sai số lớn (thường ở mức vài mét) và kém ổn định trong điều kiện đa đường truyền (multipath); các giải pháp dựa trên camera lại phụ thuộc điều kiện ánh sáng và phát sinh lo ngại về quyền riêng tư. Xuất phát từ thực tế đó, khóa luận xây dựng Hệ thống giám sát và điều khiển Gara thông minh dựa trên công nghệ AIoT, lấy định vị Ultra-Wideband (UWB) làm nền tảng nhằm đạt sai số ở mức centimet.

Khóa luận trình bày các nội dung chính sau:

- Công nghệ định vị UWB, mô hình Anchor–Tag và thuật toán Trilateration.
- Firmware cho module UWB EWT550 trên vi điều khiển ESP32-C3, truyền dữ liệu qua UART và MQTT.
- IoT Server xử lý dữ liệu, tính tọa độ, lọc nhiễu và phân phối trạng thái theo thời gian thực.
- Ứng dụng di động React Native/Expo hiển thị bản đồ và trạng thái phương tiện.
- Chatbot hỏi đáp tiếng Việt ứng dụng kỹ thuật Retrieval-Augmented Generation (RAG).
- Kiểm thử chức năng và đánh giá độ chính xác định vị trong thực nghiệm.

Để trình bày một cách hệ thống các nội dung trên, khóa luận được tổ chức thành năm chương:

- **Chương 1: Tổng quan**
- **Chương 2: Cơ sở lý thuyết**
- **Chương 3: Phương pháp đề xuất**
- **Chương 4: Kết quả thực hiện**
- **Chương 5: Kết luận và hướng phát triển**

Tác giả xin chân thành cảm ơn PGS.TS. Phan Anh Cang cùng quý thầy cô Khoa Công nghệ Thông tin đã hướng dẫn và hỗ trợ trong quá trình thực hiện khóa luận.

TÓM TẮT

Khóa luận trình bày việc nghiên cứu và xây dựng Hệ thống giám sát và điều khiển Gara thông minh dựa trên công nghệ AIoT. Hệ thống sử dụng công nghệ định vị Ultra-Wideband (UWB) kết hợp thuật toán Trilateration và bộ lọc One-Euro để xác định vị trí phương tiện trong nhà theo thời gian thực, trên nền vi điều khiển ESP32-C3 và giao thức truyền thông MQTT. Dữ liệu khoảng cách được xử lý tại máy chủ IoT (IoT Server) để tính tọa độ, làm trơn quỹ đạo và phân phối tới ứng dụng di động, nơi vị trí phương tiện được hiển thị trực quan trên bản đồ số 3D. Bên cạnh đó, hệ thống tích hợp một chatbot tiếng Việt ứng dụng kỹ thuật Tăng cường Truy xuất (Retrieval-Augmented Generation – RAG) cho phép người dùng tra cứu trạng thái bãi đỗ và vị trí phương tiện bằng ngôn ngữ tự nhiên. Hệ thống đã được hiện thực hóa hoàn chỉnh từ phần cứng đến ứng dụng di động và giao diện quản trị web; độ chính xác định vị được đánh giá theo một quy trình định lượng chuẩn hóa dựa trên các độ đo RMSE, MAE và CEP.

Từ khóa: UWB, định vị trong nhà, Trilateration, bộ lọc One-Euro, AIoT, chatbot RAG, MQTT, ESP32-C3.

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

Chương 1 giới thiệu tổng quan về đề tài: lý do chọn đề tài xuất phát từ nhu cầu thực tế trong giám sát và quản lý bãi đỗ xe; mục tiêu, đối tượng và phạm vi nghiên cứu; phương pháp nghiên cứu; ý nghĩa khoa học và thực tiễn của hệ thống đề xuất; các nghiên cứu liên quan đến định vị UWB và chatbot AI trong quản lý garage; và cấu trúc tổng thể của khóa luận.

1.1 Lý do chọn đề tài

Thứ nhất, số lượng phương tiện cần quản lý trong không gian kín ngày càng tăng. Cùng với sự gia tăng sở hữu ô tô, các bãi đỗ và gara sửa chữa phải tiếp nhận lưu lượng phương tiện lớn trong một mặt bằng giới hạn. Khi quy mô vượt một ngưỡng nhất định, việc theo dõi từng phương tiện bằng quan sát trực tiếp hoặc ghi chép thủ công trở nên kém khả thi, dễ phát sinh sai sót và mất thời gian.

Thứ hai, mô hình quản lý truyền thống không còn hiệu quả đối với gara sửa xe nội bộ. Khác với bãi đỗ công cộng, gara nội bộ vận hành theo quy trình tiếp nhận – di chuyển – sửa chữa – bàn giao, trong đó một phương tiện có thể được dịch chuyển qua nhiều vị trí (khu chờ, khoang sửa chữa, khu hoàn thiện). Các phương thức truyền thống như cảm biến đếm lượt ra–vào hay sơ đồ vị trí cố định chỉ phản ánh số lượng hoặc trạng thái tĩnh, không theo kịp sự thay đổi vị trí liên tục này, dẫn đến tình trạng thất lạc thông tin và khó truy vết phương tiện trong quá trình xử lý.

Thứ ba, nhu cầu định vị theo thời gian thực ngày càng cấp thiết. Thông tin vị trí cập nhật liên tục cho phép xác định nhanh chỗ trống, định vị lại phương tiện khi cần bàn giao, giám sát hiện trạng lắp đầy phục vụ điều phối, đồng thời lưu vết lịch sử di chuyển để đối soát và phát hiện các tình huống bất thường (đỗ sai khu vực, lưu lại quá lâu). Đây là những thông tin mà mô hình quản lý theo số lượng không thể cung cấp ở mức từng phương tiện.

Thứ tư, các công nghệ định vị phổ biến chưa phù hợp với môi trường trong nhà. Tín hiệu định vị vệ tinh (GNSS) bị che chắn bởi kết cấu bê tông nên gần như mất tác dụng; định vị bằng Wi-Fi hoặc Bluetooth Low Energy (BLE) có chi phí thấp nhưng kém ổn định và dễ bị nhiễu trong môi trường nhiều vật cản; giải pháp dựa

trên camera lại phụ thuộc điều kiện ánh sáng và phát sinh lo ngại về quyền riêng tư. Công nghệ Ultra-Wideband (UWB) khắc phục các hạn chế này nhờ nguyên lý đo thời gian truyền sóng băng siêu rộng, cho phép phân biệt tốt tín hiệu trực tiếp với tín hiệu phản xạ và đạt độ chính xác cao trong không gian kín.

Thứ năm, nhu cầu tra cứu thông tin thuận tiện hơn cho người vận hành.

Việc tổng hợp tình trạng bãi đỗ, vị trí và lịch sử phương tiện thường đòi hỏi thao tác thủ công trên nhiều màn hình. Sự phát triển của các mô hình ngôn ngữ lớn kết hợp kỹ thuật Retrieval-Augmented Generation (RAG) mở ra khả năng xây dựng một chatbot hỏi đáp tiếng Việt, cho phép truy vấn dữ liệu thực tế của hệ thống bằng ngôn ngữ tự nhiên.

Từ những lý do trên, đề tài "**Hệ thống giám sát và điều khiển Gara thông minh dựa trên công nghệ AIoT**" được thực hiện nhằm giải quyết đồng thời hai vấn đề: định vị phương tiện chính xác trong không gian trong nhà bằng công nghệ UWB, và hỗ trợ tra cứu thông tin thông qua chatbot ứng dụng trí tuệ nhân tạo.

1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát: xây dựng Hệ thống giám sát và điều khiển Gara thông minh dựa trên công nghệ AIoT, có khả năng định vị phương tiện trong nhà theo thời gian thực và hỗ trợ tra cứu thông tin bằng chatbot AI.

Mục tiêu cụ thể:

1. Nghiên cứu và triển khai giải pháp định vị trong nhà dựa trên công nghệ UWB đạt độ chính xác ở mức xen-ti-mét.
2. Thiết kế và chế tạo mô hình phần cứng Tag và Anchor sử dụng vi điều khiển ESP32-C3.
3. Xây dựng hệ thống backend (IoT Server) xử lý dữ liệu định vị theo thời gian thực và quản lý trạng thái bãi xe.
4. Phát triển ứng dụng di động hiển thị vị trí xe trên bản đồ số và hỗ trợ tìm xe.

5. Tích hợp chatbot AI thông minh ứng dụng kỹ thuật RAG (Retrieval-Augmented Generation) để hỗ trợ người dùng truy vấn trạng thái garage bằng ngôn ngữ tự nhiên.

1.3 Đối tượng và phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** công nghệ định vị vô tuyến UWB (Ultra-Wideband), thuật toán Trilateration, bộ lọc One-Euro, kiến trúc hệ thống IoT, kỹ thuật RAG và Hybrid Search trong trí tuệ nhân tạo.
- **Phạm vi nghiên cứu** được giới hạn theo ba phương diện: (i) về *nội dung* — xác định tọa độ 2D (x, y) của phương tiện (không xét độ cao z), sử dụng bốn Anchor cố định cho mỗi tầng và giao tiếp qua giao thức MQTT; (ii) về *không gian* — môi trường bãi đỗ xe/gara trong nhà, thử nghiệm trên mô hình mặt bằng một tầng; (iii) về *thời gian* — trong khuôn khổ thời gian thực hiện khóa luận tốt nghiệp năm 2026.

1.4 Phương pháp nghiên cứu

Để đạt được các mục tiêu đề ra, đề tài áp dụng các phương pháp sau:

- **Nghiên cứu lý thuyết:** Tổng hợp tài liệu khoa học về các phương pháp định vị trong nhà, các mô hình đo khoảng cách bằng sóng vô tuyến, thuật toán lọc nhiễu tín hiệu và kiến trúc mô hình ngôn ngữ lớn phục vụ xây dựng chatbot.
- **Thực nghiệm và triển khai:** Thiết kế phần cứng, lập trình firmware cho vi điều khiển nhúng, xây dựng hệ thống server xử lý dữ liệu và phát triển ứng dụng di động đa nền tảng.
- **Đánh giá và kiểm thử:** Tiến hành đo đạc thực tế tại bãi xe để đánh giá độ chính xác định vị và hiệu quả phản hồi của hệ thống chatbot AI.

1.5 Ý nghĩa khoa học và thực tiễn

Về **mặt khoa học**, đề tài hệ thống hóa và vận dụng một chuỗi kỹ thuật hiện đại cho bài toán định vị trong nhà: đo khoảng cách Two-Way Ranging trên nền UWB, giải tọa độ bằng Trilateration theo phương pháp bình phương tối thiểu, làm trơn quỹ đạo bằng bộ lọc One-Euro, kết hợp với kỹ thuật Hybrid RAG cho chatbot truy vấn

dữ liệu thời gian thực. Việc tích hợp đồng thời lớp định vị độ chính xác cao và lớp tương tác ngôn ngữ tự nhiên trong một kiến trúc AIoT thống nhất là đóng góp mang tính tổng hợp của đề tài.

Về mặt thực tiễn, hệ thống giải quyết các nhu cầu cụ thể trong vận hành gara: xác định nhanh vị trí phương tiện, giám sát hiện trạng lấp đầy, phát hiện đỗ sai quy định và hỗ trợ tra cứu thông tin bằng tiếng Việt. Thiết kế dựa trên phần cứng chi phí thấp (ESP32-C3 kết hợp module UWB giá khoảng 2 USD) và phần mềm mã nguồn mở giúp giải pháp có khả năng nhân rộng cho các bãi đỗ trong nhà ở nhiều quy mô khác nhau.

Đóng góp cụ thể của khóa luận gồm bốn điểm chính:

1. **Mẫu kiến trúc "IoT Snapshot Documents"**: phương pháp tích hợp dữ liệu IoT thời gian thực vào knowledge base vector của hệ thống RAG — snapshot trạng thái garage được embed và upsert định kỳ, cho phép chatbot trả lời các câu hỏi về trạng thái thực tế mà không cần gọi API riêng. Hiệu quả được kiểm chứng định lượng ($\text{Recall}@5 = 0,821$, $\text{MRR} = 0,962$ trên 13 câu hỏi gán nhãn tại §4.3.6).
2. **Quy trình khảo sát mặt bằng walk-and-press**: phương pháp thiết lập bản đồ tầng hầm bằng cách đi bộ với Tag UWB, không cần đo vẽ thủ công; server tự phát hiện tọa độ Anchor và dựng polygon ranh giới bãi đỗ từ dữ liệu khoảng cách thu thập trong thực địa.
3. **Kiến trúc tích hợp bất đối xứng**: IoT Server xử lý toàn bộ dữ liệu định vị thô (trilateration, lọc One-Euro) rồi đẩy kết quả sạch qua Socket.IO cho hệ thống quản lý gara bên thứ ba — hai hệ thống vận hành độc lập, không chia sẻ trạng thái nội bộ.
4. **Hệ thống AIoT end-to-end**: tích hợp liên mạch từ firmware nhúng (ESP32-C3/EWT550) qua hạ tầng MQTT/NestJS/Redis/PostgreSQL đến ứng dụng di động React Native và giao diện web quản trị, tạo thành một nền tảng hoàn chỉnh có thể triển khai thực tế.
5. **Bản đồ bãi đỗ tự học**: từ lịch sử vị trí tích lũy, hệ thống tự suy luận bố trí ô đỗ (mean-shift clustering + PCA hướng hàng) và lối đi (rasterize lưới mật độ +

gom 8-connectivity), cho phép bản đồ bãi đỗ được tinh chỉnh liên tục từ dữ liệu vận hành thực tế mà không cần đo vẽ lại thủ công — một cơ chế khép kín từ định vị đến bố trí.

1.6 Các nghiên cứu liên quan

Định vị trong nhà là lĩnh vực đã được nghiên cứu rộng rãi. Khảo sát tổng quan của Zafari và cộng sự [6] phân loại các hệ thống định vị trong nhà theo công nghệ (Wi-Fi, BLE, UWB, RFID, âm thanh, từ trường) và theo phương pháp đo (RSSI, thời gian bay, góc tới, fingerprinting), đồng thời chỉ ra rằng các phương pháp dựa trên cường độ tín hiệu (RSSI) chỉ đạt sai số ở mức vài mét do nhạy với hiện tượng đa đường. Trong nhóm công nghệ băng siêu rộng, Che và cộng sự [7] xây dựng hệ thống định vị UWB cho môi trường Internet vạn vật công nghiệp (IIoT) sử dụng thuật toán Trilateration và bộ lọc Kalman để ổn định quỹ đạo, đạt độ chính xác ở cấp xen-ti-mét, khẳng định ưu thế của đo thời gian bay so với RSSI. Tuy nhiên, hệ thống trong [7] được thiết kế cho môi trường công nghiệp có hình học cố định và không tích hợp lớp giao diện người dùng hay chatbot AI. Đề tài này mở rộng hướng tiếp cận đó cho bài toán gara: bổ sung lớp khảo sát mặt bằng tự động (walk-and-press), tích hợp ứng dụng di động và chatbot RAG thành một kiến trúc AIoT thống nhất.

Ở khâu lọc nhiễu tín hiệu định vị, hai hướng tiếp cận phổ biến là bộ lọc Kalman [12] và bộ lọc One-Euro [2]. Kalman Filter là bộ lọc tối ưu tuyến tính cho hệ thống có mô hình động học (state-space model) đã biết — phù hợp khi quỹ đạo chuyển động có thể mô hình hóa (robot, phương tiện đường trơn). One-Euro Filter [2] không yêu cầu mô hình động học: tự động điều chỉnh mức độ lọc theo tốc độ thay đổi của tín hiệu — lọc mạnh khi tín hiệu thay đổi chậm (xe đứng yên), lọc nhẹ khi tín hiệu thay đổi nhanh (xe di chuyển). Đối với bài toán bãi đỗ xe, quỹ đạo phương tiện không dự đoán được trước (có thể dừng đột ngột, vào bất kỳ ô nào) nên One-Euro phù hợp hơn: đơn giản hơn, không cần hiệu chỉnh ma trận nhiễu, và có độ trễ thấp hơn trong các thử nghiệm tương tác [2].

Công trình	Phương pháp	Ưu/Hạn chế	Khác biệt với đề tài
------------	-------------	------------	----------------------

Zafari et al. [6] (khảo sát)	Tổng hợp Wi-Fi/BLE/UWB/ RFID	Toàn diện nhưng không đề xuất hệ thống cụ thể	Đề tài triển khai hệ thống thực, không dừng ở khảo sát
Che et al. [7] (UWB-IIoT)	UWB Trilateration + Kalman filter	Chính xác cao, thiếu giao diện người dùng và AI	Thêm walk-and- press, ứng dụng di động và chatbot RAG
Lewis et al. [3] (RAG)	RAG với tài liệu tĩnh	Giảm hallucination, không có dữ liệu IoT thời gian thực	Đề tài mở rộng RAG với IoT Snapshot Documents
One-Euro vs Kalman [2][12]	Lọc tín hiệu thích nghi / tối ưu tuyến tính	One-Euro: đơn giản, độ trễ thấp; Kalman: cần mô hình động học	Đề tài chọn One- Euro vì quỹ đạo bãi đỗ không thể mô hình hóa trước

Ở lớp tương tác, kỹ thuật Retrieval-Augmented Generation do Lewis và cộng sự [3] đề xuất đã trở thành hướng tiếp cận phổ biến để chatbot truy cập tri thức bên ngoài và giảm hiện tượng "ảo giác" của mô hình ngôn ngữ lớn. Đề tài kế thừa hướng tiếp cận này và mở rộng cho dữ liệu IoT thời gian thực: thay vì chỉ truy xuất tài liệu tĩnh, hệ thống đưa trạng thái bãi đỗ và vị trí phương tiện cập nhật liên tục vào ngữ cảnh truy vấn.

1.7 Cấu trúc khóa luận

Khóa luận được tổ chức thành năm chương:

- **Chương 1 — Tổng quan đề tài:** trình bày lý do chọn đề tài, mục tiêu, đối tượng và phạm vi, phương pháp nghiên cứu, ý nghĩa khoa học – thực tiễn và các nghiên cứu liên quan.
- **Chương 2 — Cơ sở lý thuyết:** giới thiệu các công nghệ cốt lõi gồm UWB, Trilateration, bộ lọc One-Euro, vi điều khiển ESP32-C3, giao thức MQTT, NestJS, PostgreSQL/pgvector, Redis, React Native, RAG/MCP và Docker/Nginx.

- **Chương 3 — Phương pháp đề xuất:** mô tả kiến trúc tổng thể, thiết kế phần cứng – firmware, quy trình xử lý tại IoT Server, cơ chế Hybrid RAG và quy trình vận hành xe vào – xe ra.
- **Chương 4 — Kết quả thực hiện:** trình bày môi trường triển khai, các nhóm tính năng đã hiện thực hóa, phương pháp đánh giá độ chính xác định vị và phân tích dữ liệu vận hành.
- **Chương 5 — Kết luận và hướng phát triển:** tổng kết kết quả đạt được, nêu hạn chế và đề xuất hướng phát triển trong tương lai.

Kết luận chương 1. Chương 1 đã xác lập bối cảnh và động cơ của đề tài, làm rõ mục tiêu, đối tượng, phạm vi và phương pháp nghiên cứu, đồng thời nêu ý nghĩa khoa học – thực tiễn và đặt đề tài trong tương quan với các nghiên cứu liên quan. Các cơ sở lý thuyết phục vụ cho thiết kế hệ thống sẽ được trình bày trong Chương 2.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

Chương này giới thiệu các công nghệ cốt lõi được sử dụng trong hệ thống, bao gồm nguyên lý hoạt động, ưu nhược điểm và lý do lựa chọn. Các công nghệ được phân nhóm theo vai trò: lớp phần cứng IoT, lớp truyền thông, lớp xử lý backend, lớp giao diện người dùng và lớp AI.

2.1 Công nghệ định vị trong nhà Ultra-Wideband (UWB)



Hình 2.1: Công nghệ Ultra-Wideband (UWB)

Ultra-Wideband (UWB) là công nghệ vô tuyến sử dụng dải tần số rất rộng (từ 3,1–10,6 GHz, băng thông tối thiểu 500 MHz) và truyền các xung điện từ cực ngắn thay vì tín hiệu liên tục. Điểm khác biệt căn bản so với Wi-Fi hay Bluetooth là UWB đo **thời gian bay** (Time of Flight) của tín hiệu thay vì cường độ sóng. Vì tốc độ ánh sáng là hằng số đã biết, thời gian bay cho phép tính khoảng cách với độ chính xác 10–30 cm — vượt trội so với 1–5 m của các công nghệ dựa trên RSSI [6]. Trong môi trường bãi đỗ xe tầng hầm, nơi sóng GPS không xuyên qua bê tông và sóng Wi-Fi bị phản xạ nhiều từ xe kim loại, UWB là lựa chọn thực tế duy nhất đáp ứng được yêu cầu phân biệt chính xác từng ô đỗ.

2.1.1 So sánh với các công nghệ định vị khác

Bảng 2.1 so sánh ba công nghệ định vị phổ biến theo các tiêu chí quan trọng với bài toán định vị trong bãi đỗ xe trong nhà.

Bảng 2.1: So sánh các công nghệ định vị trong nhà

Tiêu chí	GPS	Wi-Fi / BLE	UWB
Nguyên lý đo	Tín hiệu vệ tinh	Cường độ sóng (RSSI)	Thời gian bay tín hiệu (ToF)
Hoạt động trong nhà	Không	Có	Có
Sai số điển hình	Không áp dụng	1 – 5 m	10 – 30 cm
Ảnh hưởng phản xạ đa đường	Cao	Rất cao	Thấp
Tiêu thụ điện	Thấp	Thấp	Thấp – Trung bình
Chi phí module	Thấp	Thấp	Trung bình (~2 USD)

2.1.2 Cách đo khoảng cách: Two-Way Ranging

Two-Way Ranging (TWR) là giao thức đo khoảng cách hai chiều giữa hai thiết bị UWB, được chuẩn hóa cùng các kỹ thuật đo khoảng cách nâng cao trong tiêu chuẩn IEEE 802.15.4z [1]. Luồng hoạt động như sau: thiết bị chủ động (Initiator) gửi một tín hiệu ping và ghi nhận thời điểm gửi; thiết bị bị động (Responder) nhận được, chờ một khoảng thời gian xử lý nội bộ cố định rồi phản hồi; Initiator đo tổng thời gian khứ hồi và tính ra khoảng cách. Ưu điểm quan trọng của TWR là **không cần đồng bộ đồng hồ** giữa hai thiết bị — điểm yếu lớn của phương pháp đo thời gian một chiều (TDoA).

Ảnh xạ vai trò trong hệ thống đề tài: Tag (module gắn trên xe, địa chỉ AB00) đóng vai trò **Initiator** theo thuật ngữ TWR — khởi động chu trình đo, thu thập khoảng cách đến tất cả Anchor trong một lần đo và gửi bản tin MQTT ``iot/{chip_id}/uwb`` về server mỗi giây. Anchor (nút cố định AA01–AA04 tại các góc bãi) đóng vai trò

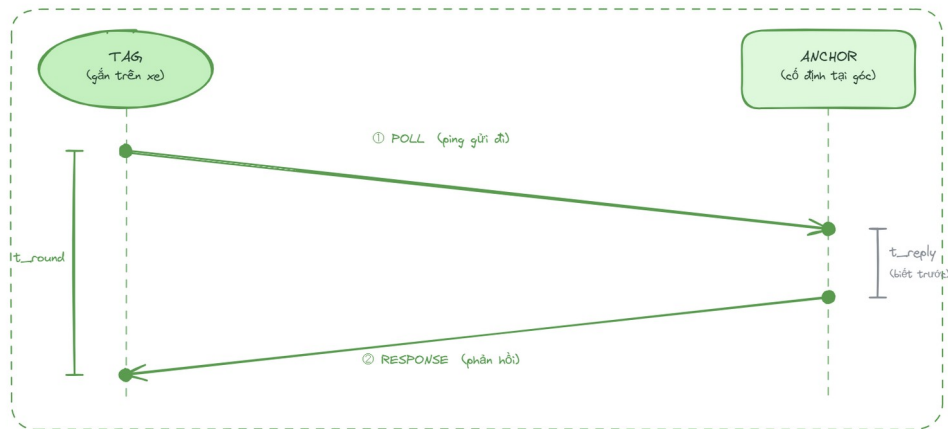
Responder — chỉ phản hồi tín hiệu từ Tag khi được poll, không tự khởi tạo đo khoảng cách và không gửi bản tin UWB độc lập lên server. Theo chuẩn EWT550, Initiator tương ứng với Base Station (ROLE=1), Responder tương ứng với Slave (ROLE=0).

Khoảng cách được tính theo công thức:

$$d = c \cdot \frac{t_{round} - t_{reply}}{2}$$

Trong đó:

- d — khoảng cách cần tính (m)
- c — tốc độ ánh sáng ($\approx 3 \times 10^8$ m/s)
- t_{round} — thời gian tổng từ lúc Initiator gửi đến lúc nhận được phản hồi (s)
- t_{reply} — thời gian Responder xử lý nội bộ trước khi phản hồi (s, đã biết trước)



Hình 2.2: Two-Way Ranging (TWR)

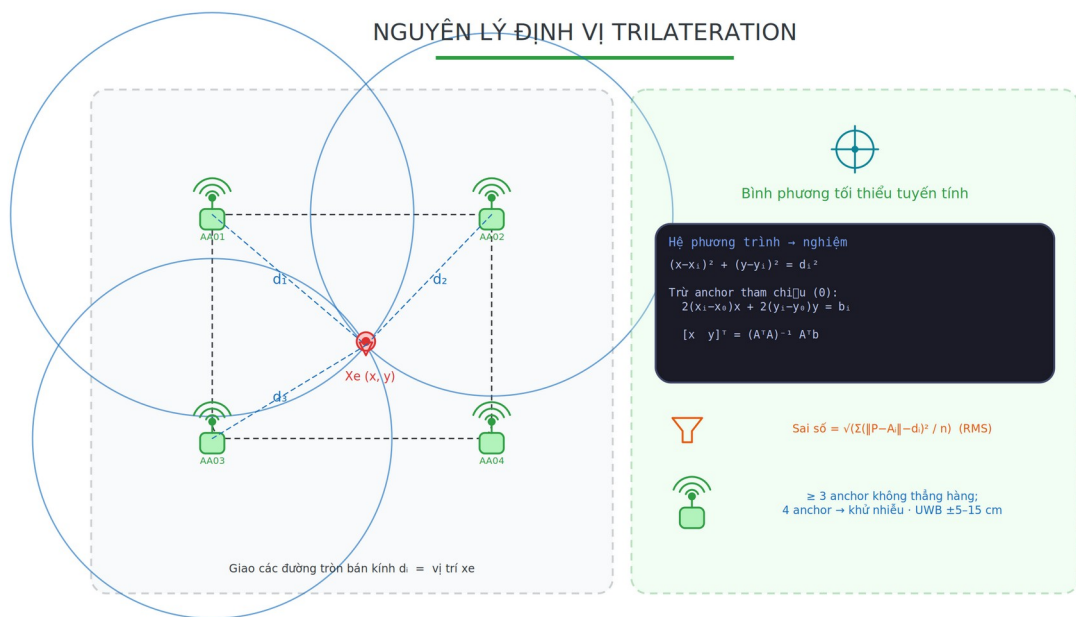
2.1.3 Xác định vị trí 2D: Trilateration

Sau khi đo được khoảng cách đến các Anchor, vị trí của Tag được xác định bằng thuật toán Trilateration. Ý tưởng: nếu biết Tag cách Anchor A_1 một khoảng d_1 , thì Tag nằm đâu đó trên đường tròn tâm A_1 bán kính d_1 . Vẽ đường tròn tương tự cho mỗi Anchor — giao điểm của tất cả các đường tròn cho vị trí duy nhất của Tag. Khi có nhiều điểm tham chiếu hơn mức tối thiểu, phương pháp Bình phương Tối thiểu (Least Squares) tìm nghiệm tốt nhất giảm thiểu tổng sai số đo:

$$p = (A^T A)^{-1} A^T b$$

Trong đó:

- p — vector tọa độ (x, y) ước tính của điểm cần tìm
- A — ma trận hệ số xây dựng từ tọa độ các điểm tham chiếu
- b — vector xây dựng từ khoảng cách đo được và tọa độ điểm tham chiếu
- $(\cdot)^T$ — phép chuyển vị ma trận
- $(\cdot)^{-1}$ — phép nghịch đảo ma trận



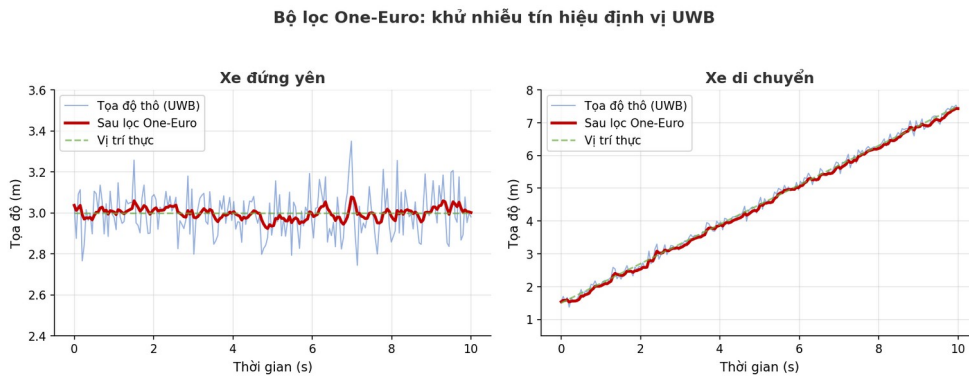
Hình 2.3: Nguyên lý định vị Trilateration

2.2 Bộ lọc One-Euro

Dù UWB chính xác hơn các công nghệ khác, tọa độ thô vẫn dao động vài chục cm mỗi giây do nhiễu đo và phản xạ sóng — khiến vị trí xe "nhảy múa" liên tục trên bản đồ dù xe đứng yên. Bộ lọc One-Euro [2] giải quyết bài toán này theo nguyên lý thích nghi: tự động điều chỉnh mức độ lọc dựa trên tốc độ thay đổi của tín hiệu.

- Khi tín hiệu thay đổi chậm (xe đứng yên): lọc mạnh → loại bỏ nhiễu
- Khi tín hiệu thay đổi nhanh (xe di chuyển): lọc nhẹ → bắt kịp chuyển động thật

So với bộ lọc Kalman [12] thường dùng trong robot và drone, One-Euro đơn giản hơn nhiều vì không cần xây dựng mô hình động học của đối tượng. Đây là lý do phù hợp cho bài toán định vị xe trong bãi đỗ, nơi quỹ đạo di chuyển không thể dự đoán trước.

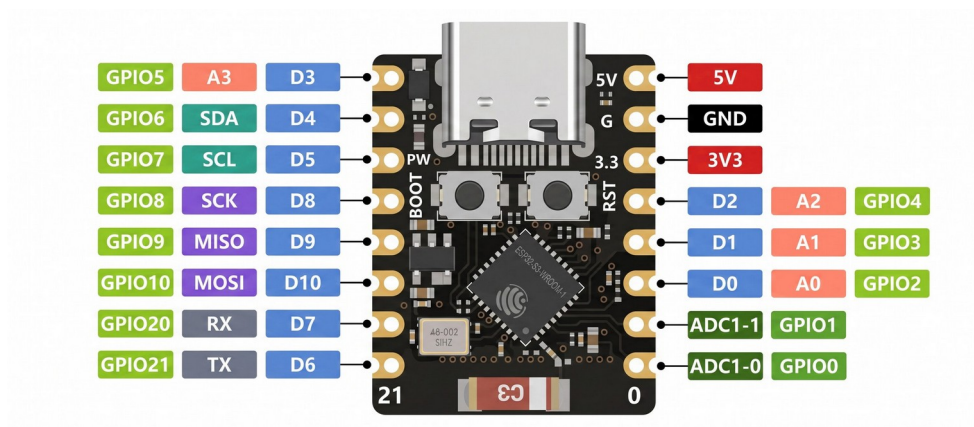


Hình 2.4: Bộ lọc One-Euro

2.3 Vi điều khiển ESP32-C3

ESP32-C3 là vi điều khiển tích hợp (SoC) do Espressif Systems (Trung Quốc) ra mắt năm 2020, là thế hệ kế tiếp của dòng ESP8266 và ESP32 nổi tiếng [13]. Espressif lựa chọn kiến trúc lõi RISC-V mã nguồn mở thay vì Xtensa trước đó, giúp tiết kiệm chi phí bản quyền và dễ tích hợp với chuỗi công cụ GCC nguồn mở. Chip tích hợp sẵn Wi-Fi 802.11 b/g/n và Bluetooth 5.0 BLE trên cùng một die, module Super Mini chỉ 18×22 mm với giá khoảng 1–2 USD.

Dòng ESP32 hiện là vi điều khiển IoT phổ biến nhất thế giới với hơn 1 tỷ chip đã xuất xưởng (tính đến 2024). Cộng đồng maker, sinh viên và kỹ sư toàn cầu đóng góp hàng nghìn thư viện, ví dụ và tutorial trên GitHub, Arduino Forum và Hackaday. ESP-IDF — framework chính thức của Espressif — cung cấp sẵn hệ điều hành thời gian thực FreeRTOS, stack Wi-Fi/BLE, cơ chế OTA và giao tiếp đa loại ngoại vi. ESP32-C3 phù hợp với các dự án IoT nhúng cần kết nối mạng, chi phí thấp và kích thước nhỏ. Hạn chế là đơn lõi CPU, không phù hợp xử lý tín hiệu nặng hay AI on-device.



Hình 2.5: ESP32-C3 Super Mini

2.4 Bluetooth Low Energy (BLE)

Bluetooth Low Energy (BLE) được giới thiệu trong chuẩn Bluetooth 4.0 năm 2010 bởi Bluetooth SIG, ban đầu mang tên Wibree (phát triển bởi Nokia từ 2006). Mục tiêu là tạo ra một chuẩn Bluetooth mới tiêu tốn điện năng thấp hơn 10–100 lần so với Bluetooth Classic, cho phép thiết bị chạy từ pin nhỏ trong hàng tháng đến hàng năm. BLE hoạt động ở dải tần 2.4 GHz, sử dụng 40 kênh 2 MHz và truyền dữ liệu theo từng gói ngắn.

Ngày nay BLE có mặt trên mọi điện thoại thông minh, thiết bị đeo và hầu hết SoC IoT. Chế độ **Advertising** cho phép thiết bị phát liên tục các gói beacon mà không cần kết nối đôi hướng — điện thoại xung quanh quét và nhận thông tin, ước lượng khoảng cách qua cường độ tín hiệu (RSSI). Đây là nền tảng của nhiều ứng dụng định vị trong nhà như Apple AirTag, Google Find My Network và Indoor GPS. Ưu điểm: tiêu thụ điện cực thấp, tích hợp rộng rãi, không cần hạ tầng phụ. Nhược điểm: định vị RSSI sai số 1–3 m, bị ảnh hưởng bởi phản xạ sóng và môi trường kim loại.

2.5 Giao thức MQTT



Hình 2.6: Logo giao thức MQTT

MQTT (Message Queuing Telemetry Transport) được phát triển bởi Andy Stanford-Clark (IBM) và Arlen Nipper (Arcom) năm 1999 để giám sát đường ống dầu qua vệ tinh bằng thông hẹp. Giao thức được IBM mở công khai năm 2010 và trở thành chuẩn OASIS năm 2014 [8]. Ngày nay MQTT là giao thức truyền thông IoT phổ biến nhất thế giới, được tích hợp trong AWS IoT, Azure IoT Hub, Google Cloud IoT và hầu hết các nền tảng IoT thương mại.

MQTT dựa trên mô hình **Publish/Subscribe**: publisher gửi bản tin lên một Topic thông qua Broker trung gian; subscriber đăng ký Topic đó và nhận bản tin tức thời. Thiết bị không cần biết địa chỉ nhau — tất cả liên lạc qua Broker. Header gói tin tối thiểu chỉ 2 byte, kết nối TCP liên tục thay vì mở/đóng mỗi lần — phù hợp cho thiết bị tài nguyên thấp gửi dữ liệu liên tục. Tính năng **Last Will and Testament (LWT)**: thiết bị đặt trước "di chúc" với Broker; nếu mất kết nối đột ngột, Broker tự gửi thông báo đến các bên liên quan — phát hiện thiết bị offline mà không cần polling. Ưu điểm: nhẹ, hỗ trợ TLS, ba mức QoS. Nhược điểm: Broker là điểm lỗi đơn (SPOF), không có cơ chế tự khám phá topic.

2.6 NestJS



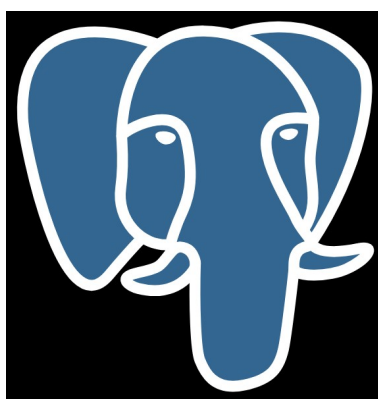
Hình 2.7: Logo NestJS

NestJS [4] được Kamil Myśliwiec phát hành năm 2017 với mục tiêu mang kiến trúc rõ ràng của Angular vào phía server Node.js. Trước NestJS, hầu hết dự án Node.js dùng Express thuần túy — linh hoạt nhưng không có quy ước kiến trúc nào, dẫn đến code khó bảo trì khi dự án mở rộng. NestJS giải quyết vấn đề này bằng kiến trúc **Module** rõ ràng: mỗi nhóm chức năng đóng gói trong một module, tách biệt hoàn toàn với các module khác. Cơ chế **Dependency Injection** tự động quản lý vòng đời

và cung cấp các phụ thuộc — lập trình viên khai báo cần gì, framework tự lo khởi tạo và truyền vào.

NestJS hiện có hơn 68 nghìn sao trên GitHub (tháng 6/2025), được dùng bởi nhiều công ty lớn như Adidas, Autodesk, Roche và các startup toàn cầu. Framework hỗ trợ đồng thời nhiều giao thức giao tiếp (HTTP REST, WebSocket, MQTT, gRPC, microservices) trong cùng một codebase — ưu điểm lớn cho hệ thống IoT cần xử lý nhiều nguồn dữ liệu. Nhược điểm: startup chậm hơn Express thuần, learning curve cao hơn với developer mới.

2.7 Cơ sở dữ liệu PostgreSQL và pgvector



Hình 2.8: Logo PostgreSQL

PostgreSQL bắt đầu là dự án nghiên cứu POSTGRES tại UC Berkeley năm 1986 dưới sự dẫn dắt của Michael Stonebraker. Sau nhiều lần tái cấu trúc, phiên bản mã nguồn mở hoàn chỉnh ra mắt năm 1996 với tên PostgreSQL. Đây là RDBMS mã nguồn mở tuân thủ chuẩn SQL nghiêm ngặt nhất, nổi bật với hệ thống extension cho phép bổ sung kiểu dữ liệu và chức năng mới mà không cần sửa code lõi.

Theo khảo sát Stack Overflow Developer Survey 2024, PostgreSQL là cơ sở dữ liệu được lập trình viên yêu thích nhất trong nhiều năm liên tiếp. Nhiều công ty lớn như Apple, Instagram, Spotify và Cisco dùng PostgreSQL ở quy mô hàng tỷ bản ghi. Extension **pgvector** (ra mắt 2021) bổ sung khả năng lưu trữ và tìm kiếm vector nhúng (embedding) — cho phép xây dựng tìm kiếm ngữ nghĩa ngay trong database quan hệ mà không cần hệ thống vector database riêng như Pinecone hay Weaviate.

Ưu điểm: ACID, mở rộng tốt, giàu tính năng. Nhược điểm: tốn tài nguyên hơn MySQL, cấu hình phức tạp hơn.

2.8 Redis

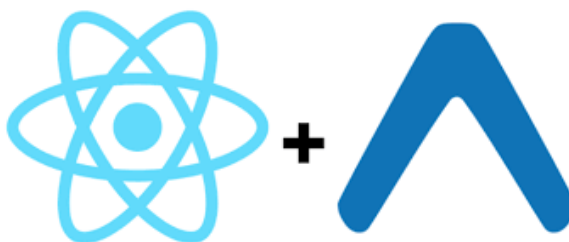


Hình 2.9: Logo Redis

Redis (Remote Dictionary Server) do Salvatore Sanfilippo phát triển năm 2009 để giải quyết bài toán đếm trang xem thời gian thực mà MySQL không đủ nhanh. Ngay từ đầu Redis chọn triết lý lưu toàn bộ dữ liệu trong RAM để đạt tốc độ tối đa, sau đó mới bổ sung tùy chọn bền vững (persistence) cho các trường hợp cần. Tốc độ đọc/ghi đạt hàng triệu thao tác mỗi giây, latency dưới 1 mili-giây — nhanh hơn 10–100 lần so với database lưu trên ổ đĩa.

Redis là in-memory database phổ biến nhất thế giới, được dùng bởi Twitter, GitHub, Snapchat, Stack Overflow và hàng triệu ứng dụng khác. Ngoài tốc độ, Redis hỗ trợ nhiều cấu trúc dữ liệu phong phú (String, Hash, List, Set, Sorted Set, Stream) và các tính năng quan trọng: TTL tự động xóa key sau thời gian định trước, Pub/Sub cho giao tiếp bất đồng bộ giữa các process. Redis phù hợp cho cache, session, hàng đợi tác vụ và lưu trạng thái thời gian thực. Nhược điểm: dữ liệu có thể mất nếu server tắt đột ngột; RAM đắt hơn ổ đĩa nên không phù hợp lưu trữ lớn lâu dài.

2.9 React Native và Expo



Hình 2.10: Logo React Native và Expo

React Native được Facebook (nay là Meta) phát triển nội bộ từ năm 2013 và mã nguồn mở vào năm 2015, xuất phát từ hackathon "React JS for native mobile" do Jordan Walke khởi xướng. Thay vì WebView như Cordova/Ionic, React Native dịch các component React trực tiếp sang widget native của từng nền tảng (UIKit/SwiftUI trên iOS, Android View trên Android) — cho hiệu năng giao diện gần với ứng dụng viết native, trong khi vẫn dùng một bộ code TypeScript chung.

React Native hiện là framework di động đa nền tảng phổ biến thứ hai sau Flutter, được dùng bởi Microsoft, Shopify, Discord, Coinbase và hàng nghìn công ty khác. Expo SDK là lớp bổ sung cung cấp API thống nhất cho camera, thông báo đẩy, lưu trữ bảo mật và OTA update (đẩy bản cập nhật JavaScript trực tiếp đến người dùng mà không cần phát hành lại qua app store). React Native phù hợp khi cần ứng dụng đa nền tảng với trải nghiệm native, đội nhỏ và cập nhật nhanh. Nhược điểm: hiệu năng kém hơn native thuần cho đồ họa nặng; một số tính năng hệ thống cần viết native module riêng.

2.10 Server-Sent Events (SSE) và Socket.IO

Server-Sent Events (SSE) được W3C chuẩn hóa năm 2006 như một giải pháp đơn giản hơn WebSocket cho luồng dữ liệu một chiều từ server xuống client. SSE dùng kết nối HTTP thông thường được giữ mở — server gửi từng sự kiện dạng văn bản theo định dạng ``data: ...\n\n``, client tự động reconnect khi mất kết nối. Không cần thêm thư viện phía client; trình duyệt và nhiều HTTP client hỗ trợ sẵn qua EventSource API. Phù hợp cho luồng thông báo, cập nhật trạng thái và streaming AI response. Ưu điểm: đơn giản, hoạt động qua HTTP/2, proxy-friendly. Nhược điểm: chỉ một chiều, chỉ văn bản UTF-8.

Socket.IO ra mắt năm 2010 bởi Guillermo Rauch, xây trên WebSocket nhưng bổ sung nhiều tính năng: tự động fallback về HTTP long-polling, phòng (room) cho nhóm client, namespace cho phân luồng logic, acknowledgement xác nhận nhận bản tin, và reconnect tự động với backoff. WebSocket chuẩn (RFC 6455, 2011) cũng được các framework hiện đại tích hợp trực tiếp. Socket.IO phù hợp cho giao tiếp hai chiều thời gian thực như chat, game và đồng bộ trạng thái giữa nhiều client. Ưu điểm:

hai chiều, hỗ trợ binary, fallback tốt. Nhược điểm: nặng hơn SSE và WebSocket thuần, cần client library riêng.

2.11 Kỹ thuật Hybrid RAG (Retrieval-Augmented Generation)

2.11.1 Giới thiệu RAG và hạn chế của RAG thuần vector

RAG (Retrieval-Augmented Generation) được giới thiệu bởi Lewis và cộng sự (Facebook AI, 2020) như một giải pháp cho hiện tượng "hallucination" của LLM — mô hình bịa thông tin khi không có kiến thức thực tế. Ý tưởng cốt lõi: trước khi gửi câu hỏi cho LLM, hệ thống **truy xuất** (Retrieve) các tài liệu liên quan từ kho dữ liệu, **tăng cường** (Augment) chúng vào prompt, rồi mới để LLM **sinh** (Generate) câu trả lời dựa trên thông tin thực tế vừa cung cấp thay vì kiến thức tĩnh từ lúc huấn luyện. Nhờ đó RAG khắc phục hai hạn chế lớn của LLM thuần: giảm hiện tượng bịa đặt nhờ có dữ liệu thực làm căn cứ, và bổ sung được tri thức nội bộ, cập nhật liên tục mà mô hình không được huấn luyện sẵn.



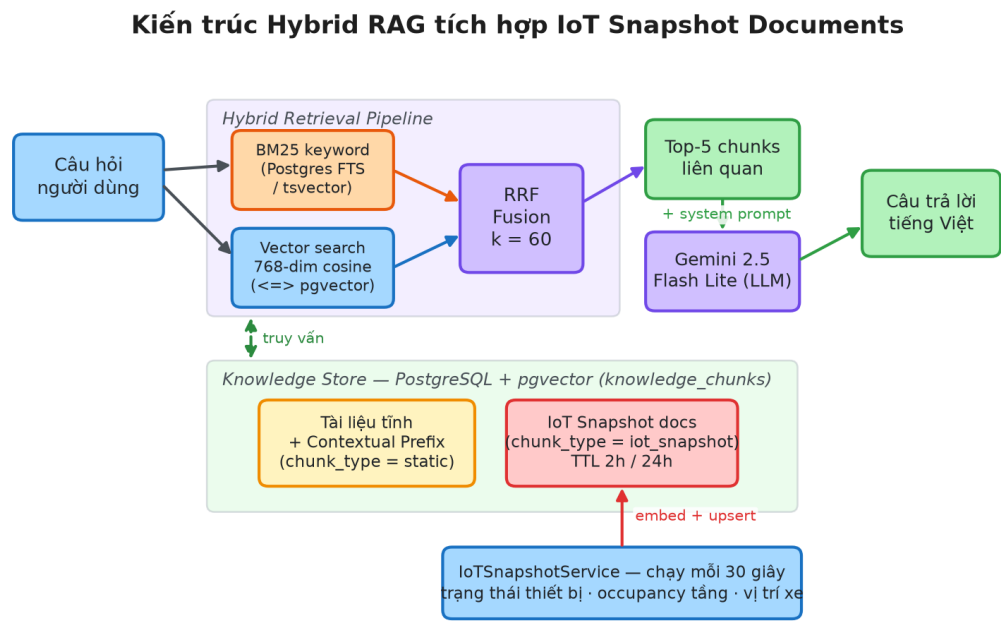
Hình 2.11: Mô hình Retrieval-Augmented Generation (RAG)

Tuy nhiên, RAG thuần vector (chỉ dùng cosine similarity giữa các embedding) bộc lộ một điểm yếu: tìm kiếm theo ngữ nghĩa nhưng bỏ sót các truy vấn chứa từ khóa kỹ thuật chính xác. Ví dụ câu hỏi "Thiết bị AA03 có đang offline không?" chứa mã thiết bị "AA03" vốn không mang ngữ nghĩa rõ ràng trong không gian vector, khiến embedding khó tìm đúng đoạn văn bản chứa từ khóa này. Benchmark BEIR cho thấy BM25 — phương pháp tìm kiếm từ khóa cổ điển — vượt trội pure vector

trên nhiều tập dữ liệu chuyên ngành nhiều thuật ngữ. Để khắc phục, đề tài triển khai **Hybrid RAG**: kết hợp BM25 và vector search theo phương pháp Reciprocal Rank Fusion (RRF), đồng thời tích hợp **IoT Snapshot Documents** để cung cấp trạng thái hệ thống thời gian thực cho LLM.

2.11.2 Kiến trúc Hybrid RAG trong hệ thống

Hệ thống RAG của đề tài gồm hai thành phần chính: **Knowledge Store** (kho tri thức) lưu trữ và đánh chỉ mục tài liệu, và **Retrieval Pipeline** (pipeline truy xuất) xử lý mỗi câu hỏi qua hai kênh tìm kiếm song song rồi hợp nhất kết quả. Hình 2.12 minh họa toàn bộ kiến trúc cùng vòng cập nhật dữ liệu IoT thời gian thực.



Hình 2.12: Kiến trúc Hybrid RAG tích hợp IoT Snapshot Documents

Knowledge Store lưu hai loại document trong cùng một bảng knowledge_chunks trên PostgreSQL + pgvector, được phân loại theo cột chunk_type như trình bày ở Bảng 2.2.

Loại tài liệu	chunk_type	Nguồn	TTL	Ví dụ nội dung
Tài liệu tĩnh	static	Markdown mô tả hệ thống	Vĩnh viễn	"UWB EWT550 độ

				chính xác ±10 cm..."
Snapshot IoT	iot_snapshot	IoTSnapshotService (5 phút)	2 giờ	"[SNAPSHOT 09:32] Tầng B1: 12 slot trống"
Cảnh báo IoT	iot_snapshot	IoTSnapshotService	24 giờ	"[SNAPSHOT 09:35] Thiết bị AA03 offline"

Bảng knowledge_chunks gồm các trường chính: định danh, nguồn và tiêu đề đoạn, nội dung văn bản, vector embedding 768 chiều, loại chunk (static hoặc iot_snapshot), tầng liên quan, thời điểm snapshot, và cột search_vector kiểu TSVECTOR được PostgreSQL tự sinh từ nội dung (GENERATED ALWAYS AS). Bảng có index GIN trên search_vector phục vụ tìm kiếm BM25 và index trên cặp (chunk_type, snapshot_ts) phục vụ truy vấn snapshot theo thời gian.

2.11.3 pgvector — Vector Database trên PostgreSQL

Vector database trong đề tài được triển khai bằng **pgvector** — extension mã nguồn mở cho PostgreSQL, cho phép lưu embedding dưới dạng cột VECTOR(768) và tìm kiếm nearest-neighbor bằng toán tử cosine distance. Bảng 2.3 so sánh pgvector với một số vector database phổ biến.

Vector DB	Loại	Ưu điểm	Hạn chế
pgvector	Extension PostgreSQL	Dùng chung CSDL sẵn có, SQL quen thuộc	Hiệu năng kém DB chuyên biệt khi dataset rất lớn
Pinecone	Cloud-native	Hiệu năng cao, dạng managed	Chi phí cao, phụ thuộc cloud
Chroma	Standalone	Nhẹ, dễ cài đặt	Ít tính năng enterprise
Weaviate	Standalone	Nhiều tính năng	Vận hành phức tạp

pgvector được lựa chọn vì hệ thống đã dùng PostgreSQL, tránh phải vận hành thêm một service riêng, phù hợp quy mô dataset nhỏ đến vừa của một garage doanh nghiệp. Mô hình embedding sử dụng là gemini-embedding-001 của Google, sinh vector 768 chiều, tách biệt hoàn toàn với mô hình sinh văn bản Gemini 2.5 Flash Lite.

2.11.4 Hybrid Search — BM25, pgvector và Reciprocal Rank Fusion

Retrieval Pipeline xử lý mỗi câu hỏi qua hai kênh song song. **Kênh BM25 (tìm theo từ khóa)** [10] dùng Full-Text Search của PostgreSQL với tsvector và plainto_tsquery, xếp hạng tài liệu theo tần suất và tầm quan trọng của từ khóa — đặc biệt hiệu quả với các thuật ngữ kỹ thuật như "AA03", "AB00", "EWT550", "tầng B1". **Kênh Vector (tìm theo ngữ nghĩa)** chuyển câu hỏi thành vector 768 chiều bằng gemini-embedding-001 rồi lấy top-20 tài liệu gần nhất theo cosine distance — hiệu quả với truy vấn ngữ nghĩa như "xe đang đỗ ở đâu", "tầng nào còn chỗ".

Hai danh sách kết quả được hợp nhất bằng **Reciprocal Rank Fusion (RRF)** với hằng số $k = 60$ (giá trị chuẩn theo tài liệu gốc của Cormack và cộng sự [9]):

$$\text{RRF}(d) = \sum_r \frac{1}{60 + r(d)}$$

Trong đó:

- d — tài liệu ứng viên
- $r(d)$ — thứ hạng của tài liệu d trong danh sách kết quả r (đánh số từ 1)
- Tổng được tính trên cả hai danh sách kết quả (BM25 và vector)

Tài liệu xuất hiện ở thứ hạng cao trong cả hai danh sách sẽ nhận điểm RRF cao nhất. Toàn bộ truy vấn được thực hiện bằng một câu lệnh SQL duy nhất gồm ba bước: một CTE xếp hạng top-20 theo cosine distance, một CTE xếp hạng top-20 theo ts_rank của full-text search, và một FULL OUTER JOIN hai CTE để tính điểm RRF rồi lấy top-5. Bộ lọc thời gian (chunk_type = static, hoặc snapshot_ts nằm trong vòng 2 giờ) được áp ở cả hai CTE để loại các snapshot IoT cũ không còn phản ánh đúng trạng thái thực tế.

2.11.5 IoT Snapshot Documents — dữ liệu thời gian thực trong Knowledge Base

Đây là đóng góp kỹ thuật chính của đề tài trong lĩnh vực RAG: **tích hợp dữ liệu IoT thời gian thực vào knowledge base vector** theo mô hình snapshot document. Thay vì chỉ lưu tài liệu tĩnh, hệ thống liên tục sinh các "knowledge document" mô tả trạng thái thực tế của garage rồi upsert vào cùng kho pgvector. Dịch vụ IoTSnapshotService chạy mỗi 5 phút (288 lần/ngày, dưới hạn mức embedding 1.000 lần/ngày) và tạo ba loại snapshot: (1) trạng thái thiết bị — so sánh danh sách thiết bị online trong Redis với toàn bộ board trong cơ sở dữ liệu, ví dụ `[SNAPSHOT 09:32] Online: AA01, AA02, AA04, AB00; Offline: AA03`, kèm một snapshot cảnh báo riêng (TTL 24 giờ) khi có thiết bị offline; (2) mức lấp đầy từng tầng — đếm số xe hiện diện trong 5 phút gần nhất, ví dụ `[SNAPSHOT 09:32] Tầng B1: tổng 30 slot, đã đỗ 18, còn trống 12`; (3) vị trí xe — vị trí mới nhất của từng xe trong 60 giây gần nhất, ví dụ `Xe AB00 ở tầng B1, tọa độ (12.35 m, 8.52 m), độ chính xác 0.11 m`.

Mỗi snapshot được embed thành vector và upsert vào knowledge_chunks. Khi người dùng hỏi "Tầng B1 còn bao nhiêu chỗ?", pipeline tìm đúng snapshot occupancy của tầng B1 nhờ cả BM25 (từ khóa "B1") lẫn vector search (ngữ nghĩa "còn chỗ trống"), giúp LLM trả lời dựa trên dữ liệu thực. Một job cleanup chạy mỗi 10 phút xóa các snapshot thường cũ hơn 2 giờ và snapshot cảnh báo cũ hơn 24 giờ, giữ kích thước bảng ổn định và bảo đảm LLM không nhận dữ liệu lỗi thời.

2.11.6 Contextual Prefix — nâng cao chất lượng embedding tài liệu tĩnh

Trước khi embed mỗi đoạn tài liệu tĩnh, hệ thống thêm một dòng prefix mô tả nguồn gốc của đoạn — ví dụ `[Tài liệu: garage-system — uwb-technology]` đặt trước nội dung. Kỹ thuật này dựa trên nghiên cứu Contextual Retrieval của Anthropic [14], cho thấy việc thêm ngữ cảnh vào từng chunk trước khi embed giúp giảm đáng kể tỷ lệ chunk không truy xuất được so với embedding văn bản thuần, do mô hình embedding hiểu rõ hơn đoạn đang nói về chủ đề gì trong tài liệu nào. Riêng snapshot IoT không cần prefix này vì dòng đầu `[SNAPSHOT ...]` đã tự cung cấp ngữ cảnh thời gian và loại thông tin.

2.11.7 Tích hợp RAG vào Chatbot

Trong IoT Server, RAG được thực hiện ngay trước khi gọi Gemini: hệ thống gọi hybridSearch để lấy 5 chunk liên quan nhất, nối chúng vào system prompt dưới mục "Thông tin liên quan từ knowledge base", rồi gửi prompt đã bổ sung cùng câu hỏi tới Gemini. Trong vòng lặp agentic, nếu cần dữ liệu cụ thể hơn (ví dụ vị trí chính xác của một xe), LLM sẽ gọi MCP tool tương ứng. Hai lớp này bổ sung cho nhau: RAG cung cấp **ngữ cảnh tổng quan**, còn MCP tool cung cấp **dữ liệu cụ thể theo yêu cầu**. Bảng 2.4 tóm tắt ba phương pháp RAG được thiết kế để thực nghiệm so sánh ở Chương 4.

Phương pháp	BM25	Vector Search	IoT Snapshot	Contextual Prefix
Baseline (vanilla RAG)	Không	Có	Không	Không
Advanced-A (Hybrid RAG)	Có	Có	Có	Không
Advanced-B (Hybrid + Contextual)	Có	Có	Có	Có

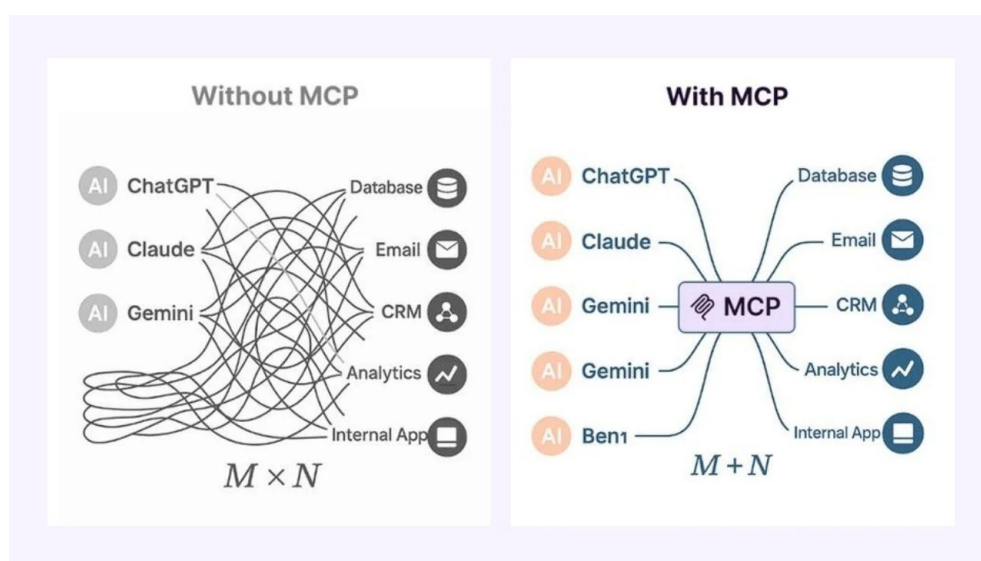
Phương pháp Advanced-B là thiết kế được triển khai trong sản phẩm cuối; phương pháp Baseline được giữ lại để phục vụ thực nghiệm đối chứng trong luận văn.

Để kiểm chứng hiệu quả thực tế của thiết kế Advanced-B so với Baseline, khóa luận xây dựng một harness đánh giá retrieval offline độc lập với pipeline vận hành. Phương pháp đánh giá bao gồm: (i) tập câu hỏi tiếng Việt về trạng thái gara (vị trí xe, số chỗ trống, thiết bị offline...) cùng câu trả lời chuẩn được xác định trước; (ii) chạy ba phương pháp (Baseline / Advanced-A / Advanced-B) độc lập trên cùng tập câu hỏi; (iii) đo **Recall@k** (tỉ lệ chunk liên quan nằm trong top-k kết quả truy xuất), **nDCG@k** (chất lượng xếp hạng), **latency trung bình** mỗi lần truy vấn, và chạy **ablation có/không TTL** để định lượng vai trò của cơ chế làm cũ snapshot IoT. Kết quả đo (chi tiết tại §4.3.6) xác nhận vai trò của truy hồi ngữ nghĩa: trên bộ 13 câu hỏi

gán nhãn, cấu hình Hybrid đạt **Recall@5 = 0,821** và **MRR = 0,962**, trong khi BM25 thuần chỉ đạt 0,026 — chênh lệch hơn 30 lần.

2.12 Giao thức MCP (Model Context Protocol)

MCP (Model Context Protocol) [5] được Anthropic công bố cuối năm 2024 như một chuẩn mở cho phép LLM gọi các công cụ (tool) bên ngoài theo cách nhất quán. Trước MCP, mỗi hệ thống AI tích hợp tool theo cách riêng; MCP chuẩn hóa giao thức: LLM sinh yêu cầu gọi tool dạng JSON có cấu trúc → MCP Server thực thi → trả kết quả về. Tool và LLM tách biệt hoàn toàn — cùng một bộ tool có thể dùng với bất kỳ LLM nào hỗ trợ MCP. MCP nhanh chóng được hỗ trợ bởi Claude (Anthropic), OpenAI, Google và nhiều IDE như VS Code, Cursor. Ưu điểm: chuẩn hóa, tái sử dụng, dễ mở rộng. Nhược điểm: thêm latency một hop; chuẩn còn mới, đang tiếp tục phát triển.



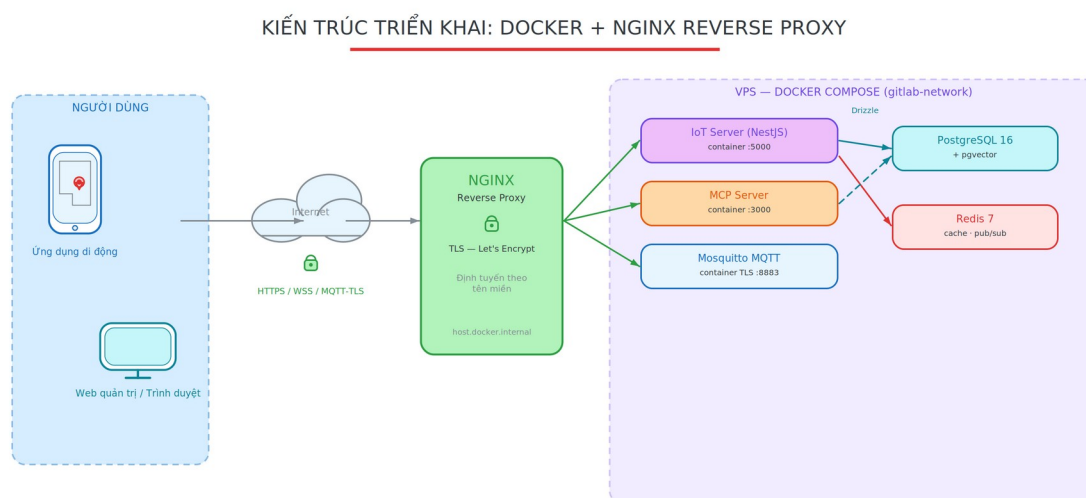
Hình 2.13: Model Context Protocol (MCP)

2.13 Docker và Nginx

Docker được Solomon Hykes giới thiệu tại PyCon 2013, mã nguồn mở ngay năm đó. Docker giải quyết vấn đề kinh điển "chạy được trên máy tôi nhưng lỗi trên server" bằng cách đóng gói ứng dụng cùng toàn bộ runtime, thư viện và cấu hình vào một container độc lập. Container chạy nhất quán trên mọi máy có Docker engine, không phụ thuộc hệ điều hành host. Docker thay đổi hoàn toàn cách triển khai phần

mềm: theo khảo sát Stack Overflow 2024, Docker là công cụ DevOps được dùng nhiều nhất toàn cầu. **Docker Compose** cho phép định nghĩa hệ thống đa service trong một file YAML và khởi động tất cả bằng một lệnh — lý tưởng cho môi trường phát triển và triển khai trên VPS. Ưu điểm: nhất quán môi trường, dễ scale, cô lập service. Nhược điểm: container dùng chung kernel host, bảo mật kém hơn VM.

Nginx (phát âm "engine-x") được Igor Sysoev phát triển từ 2002 để giải quyết vấn đề C10K — xử lý 10.000 kết nối đồng thời trên một server, điều Apache httpd không làm được lúc bấy giờ. Nginx dùng kiến trúc event-driven bất đồng bộ: một process có thể xử lý hàng chục nghìn kết nối mà không cần mỗi kết nối một thread riêng. Nginx hiện phục vụ khoảng 30–40% trong số 1 tỷ website hàng đầu thế giới. Vai trò reverse proxy của Nginx trong hệ thống phân tán: đứng trước nhiều backend service, nhận request từ internet, định tuyến dựa trên tên miền hay đường dẫn, xử lý TLS, load balancing và cache tĩnh. Ưu điểm: nhẹ, throughput cao, cấu hình linh hoạt. Nhược điểm: cú pháp cấu hình có learning curve; một số module cần biên dịch từ nguồn.



Hình 2.14: Kiến trúc triển khai Docker + Nginx reverse proxy

2.14 Lịch sử và các giai đoạn phát triển của mô hình ngôn ngữ lớn (LLM)

Mô hình ngôn ngữ lớn (Large Language Model — LLM) là nền tảng của thành phần chatbot trong đề tài. Để lý giải tại sao các kỹ thuật như RAG và MCP (trình bày ở §2.11–2.12) lại cần thiết, phần này điểm qua sáu giai đoạn phát triển chính của

LLM — từ mô hình thống kê sơ khai đến hệ thống đa phương thức và agentic hiện nay.

2.14.1 Mô hình ngôn ngữ thống kê (trước 2013)

Giai đoạn đầu của xử lý ngôn ngữ tự nhiên (NLP) dựa trên mô hình **n-gram thống kê**: ước lượng xác suất của từ tiếp theo dựa trên $n-1$ từ đứng trước. Mô hình n-gram đơn giản, hiệu quả với văn bản ngắn và dễ hiểu về lý thuyết, nhưng bị giới hạn bởi "cửa sổ ngữ cảnh" cứng (thường $n \leq 5$): chúng không thể nắm bắt phụ thuộc xa trong câu và vấp phải vấn đề bùng nổ không gian trạng thái khi n tăng. Phương pháp làm mượt (Kneser-Ney, Witten-Bell) giảm nhẹ vấn đề thừa dữ liệu nhưng không giải quyết triệt để giới hạn ngữ cảnh ngắn.

2.14.2 Mạng nơ-ron hồi quy — RNN và LSTM (2013–2017)

Mạng nơ-ron hồi quy (RNN) khắc phục giới hạn cửa sổ cứng bằng cách duy trì trạng thái ẩn tuần tự — mỗi bước đọc một từ và cập nhật "bộ nhớ" theo chuỗi. Tuy nhiên RNN thuần gặp vấn đề **gradient biến mất** (vanishing gradient): tín hiệu huấn luyện suy giảm theo chiều dài chuỗi, khiến mô hình quên phụ thuộc xa. **LSTM (Long Short-Term Memory)** của Hochreiter & Schmidhuber (1997) và cải tiến **GRU** (2014) giải quyết vấn đề này bằng cơ chế cổng (gate) kiểm soát luồng thông tin vào/ra bộ nhớ ô. LSTM được ứng dụng rộng rãi trong dịch máy (sequence-to-sequence), nhận dạng giọng nói và phân tích cảm xúc giai đoạn 2013–2017. Nhược điểm còn lại: xử lý tuần tự nên khó song song hoá; khi chuỗi quá dài (hàng trăm token) hiệu quả vẫn giảm sút.

2.14.3 Kiến trúc Transformer và cơ chế Self-Attention (2017)

Năm 2017, Vaswani và cộng sự (Google Brain) công bố bài báo "**Attention is All You Need**", giới thiệu kiến trúc **Transformer** — bước ngoặt định hình toàn bộ LLM hiện đại. Thay vì xử lý tuần tự, Transformer dùng cơ chế **self-attention đa đầu (multi-head self-attention)**: mỗi token trong chuỗi tính toán *điểm chú ý* với mọi token khác, thu được biểu diễn phụ thuộc ngữ cảnh toàn cục trong một bước tính song song. Kiến trúc Transformer gồm khối Encoder (mã hoá) và Decoder (giải mã), mỗi khối gồm lớp self-attention và mạng feed-forward với kết nối tắt (residual) và chuẩn

hoá lớp (layer normalization). Ưu điểm then chốt: song song hoá toàn bộ trên GPU, nắm bắt phụ thuộc xa tuyến tính, dễ mở rộng quy mô. Transformer trở thành kiến trúc nền tảng cho mọi LLM từ BERT, GPT đến Gemini và DeepSeek.

2.14.4 Tiền huấn luyện quy mô lớn — BERT và dòng GPT (2018–2020)

Kết hợp Transformer với **tiền huấn luyện tự giám sát** (self-supervised pretraining) trên corpus văn bản khổng lồ tạo ra bước nhảy vọt về hiệu năng. **BERT** (Devlin et al., Google, 2018) dùng kiến trúc Encoder hai chiều, tiền huấn luyện với hai nhiệm vụ: Masked Language Model (dự đoán từ bị che) và Next Sentence Prediction. BERT đạt kết quả state-of-the-art trên 11 tác vụ NLP cùng lúc khi công bố. Song song đó, OpenAI phát triển dòng **GPT (Generative Pre-trained Transformer)** theo hướng Decoder tự hồi quy (GPT-1, 2018 → GPT-2, 2019 → GPT-3, Brown et al., 2020): mô hình chỉ tiên đoán token tiếp theo, nhưng khi được huấn luyện trên 570 GB văn bản đa dạng với 175 tỷ tham số, GPT-3 thể hiện khả năng **few-shot learning** ấn tượng — thực hiện nhiều tác vụ mà không cần tinh chỉnh chuyên biệt.

2.14.5 Scaling Laws, Instruction Tuning và RLHF (2021–2022)

Kaplan et al. (OpenAI, 2020) chứng minh **scaling laws**: hiệu năng LLM tăng theo hàm lũy thừa (power law) của quy mô mô hình, kích thước dữ liệu và ngân sách tính toán — tạo nền tảng lý thuyết cho cuộc đua mở rộng quy mô. Tuy nhiên mô hình lớn không mặc nhiên làm theo chỉ dẫn của người dùng; **Instruction Tuning** (tinh chỉnh với tập lệnh đa dạng) giải quyết khoảng cách này. Quan trọng hơn, **RLHF — Reinforcement Learning from Human Feedback** (Ouyang et al., OpenAI, 2022, được gọi là InstructGPT/ChatGPT) huấn luyện mô hình phần thưởng (reward model) từ sở thích của người đánh giá con người, sau đó dùng thuật toán PPO để điều chỉnh LLM sinh ra câu trả lời được xếp hạng cao hơn. ChatGPT (dựa trên GPT-3.5 với RLHF) ra mắt tháng 11/2022 và đạt 100 triệu người dùng sau 2 tháng, đánh dấu LLM bước vào ứng dụng đại chúng.

2.14.6 Giai đoạn hiện nay: RAG, Agentic Tool-Calling và Đa phương thức (2023–nay)

Dù mạnh, LLM vẫn mang hai hạn chế căn bản: (i) **kiến thức tĩnh** — tham số cố định sau huấn luyện, không biết sự kiện mới hoặc dữ liệu nội bộ; (ii) **ảo giác (hallucination)** — sinh văn bản hoàn toàn mạch lạc nhưng sai thực tế. **RAG (Retrieval-Augmented Generation)** (Lewis et al., Facebook AI, 2020) giải quyết hạn chế đầu bằng cách gắn pipeline truy xuất tài liệu vào trước bước sinh: câu hỏi được dùng để tìm kiếm ngữ cảnh liên quan từ kho tri thức ngoài, sau đó ngữ cảnh này được chèn vào prompt, giúp LLM trả lời dựa trên dữ liệu cập nhật và nguồn có thể trích dẫn. **Agentic tool-calling** (function calling) cho phép LLM phát ra lệnh gọi hàm có cấu trúc (JSON), thực thi qua API/MCP và nhận kết quả để tiếp tục suy luận — biến LLM thành "tác tử" có thể tương tác với hệ thống bên ngoài. Năm 2023–2024 chứng kiến sự bùng nổ của **mô hình đa phương thức** (multimodal): GPT-4V, Gemini 1.5, Claude 3 xử lý đồng thời văn bản, hình ảnh, âm thanh và video trong một mô hình thống nhất.

Liên hệ với đề tài: Chatbot trong hệ thống này vận hành theo mô hình hiện đại nhất của giai đoạn 2.14.6: kết hợp **Hybrid RAG** (§2.11) để tra cứu tri thức tĩnh và trạng thái IoT thời gian thực, cùng **MCP tool-calling** (§2.12) để thực thi các truy vấn có cấu trúc qua các API Garage; sử dụng **cơ chế dự phòng nhà cung cấp (provider fallback) tự động**: mô hình chính là **Gemini 2.5 Flash Lite** (gói miễn phí 500 yêu cầu/ngày) — khi gặp lỗi vượt hạn mức (429) hoặc lỗi máy chủ (5xx), hệ thống tự động chuyển sang **DeepSeek** làm dự phòng mà người dùng không cảm nhận được gián đoạn. Toàn bộ cơ chế được điều khiển qua biến môi trường, không cần thay đổi mã nguồn.

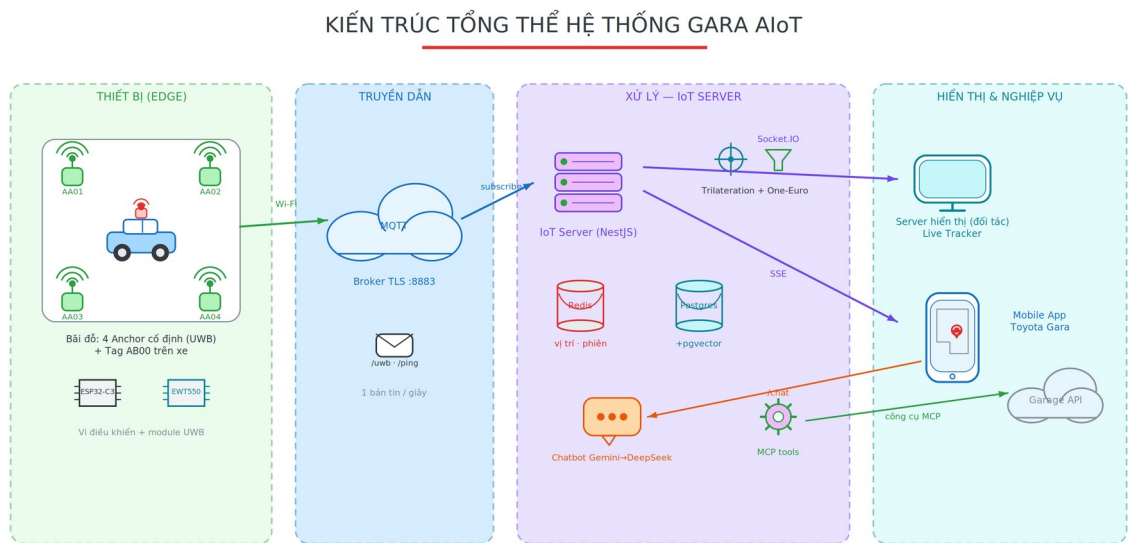
Kết luận chương 2. Chương 2 đã trình bày cơ sở lý thuyết của các công nghệ cốt lõi được sử dụng trong hệ thống: lớp định vị (UWB, Two-Way Ranging, Trilateration, bộ lọc One-Euro), lớp phần cứng (ESP32-C3, BLE), lớp truyền thông (MQTT, SSE/Socket.IO), lớp xử lý và lưu trữ (NestJS, PostgreSQL/pgvector, Redis), lớp giao diện (React Native/Expo), lớp trí tuệ nhân tạo (RAG, MCP, lịch sử phát triển LLM) và hạ tầng triển khai (Docker, Nginx). Mỗi công nghệ đều được nêu rõ nguyên

lý, ưu – nhược điểm và lý do lựa chọn, làm nền tảng để đề xuất kiến trúc và phương pháp ở Chương 3.

CHƯƠNG 3: PHƯƠNG PHÁP ĐỀ XUẤT

3.1 Mô tả tổng quan hệ thống đề xuất

Để giải quyết bài toán giám sát và quản lý bãi đỗ xe thông minh, khóa luận đề xuất một kiến trúc AIoT hoàn chỉnh gồm bốn thành phần chính liên kết với nhau: phần cứng UWB thu thập dữ liệu khoảng cách, backend IoT Server xử lý và phân tích, MCP Server cầu nối với công cụ AI, và ứng dụng di động làm giao diện người dùng. Điểm đặc trưng của kiến trúc đề xuất là hệ thống **Advanced Hybrid RAG** tích hợp dữ liệu IoT thời gian thực vào knowledge base.

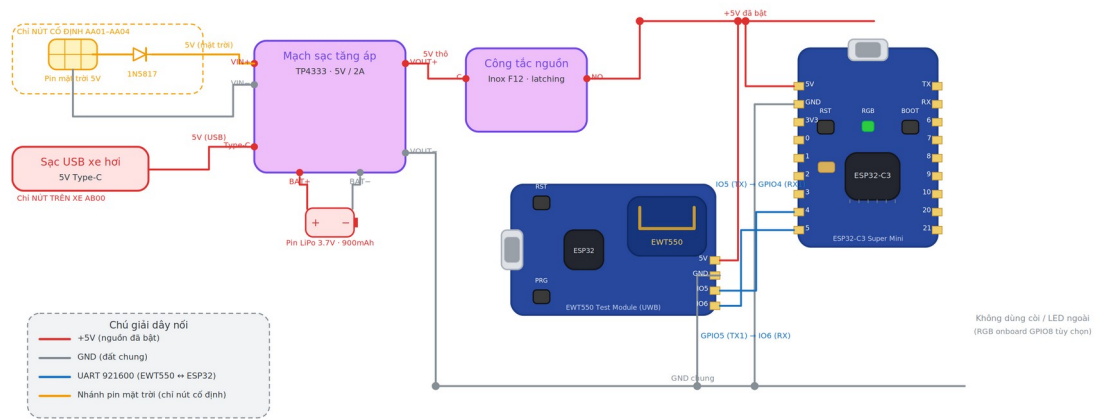


Hình 3.1: Kiến trúc tổng thể hệ thống AIoT

3.2 Thiết kế phần cứng và Firmware

Hệ thống sử dụng vi điều khiển ESP32-C3 kết nối với module UWB EWT550 qua giao tiếp UART. Các Anchor được cấu hình cố định tại các góc của bãi xe, trong khi Tag được gắn trên phương tiện. Firmware trên Tag thực hiện chu trình đo TWR liên tục và đóng gói dữ liệu khoảng cách gửi về Server qua giao thức MQTT bảo mật (TLS).

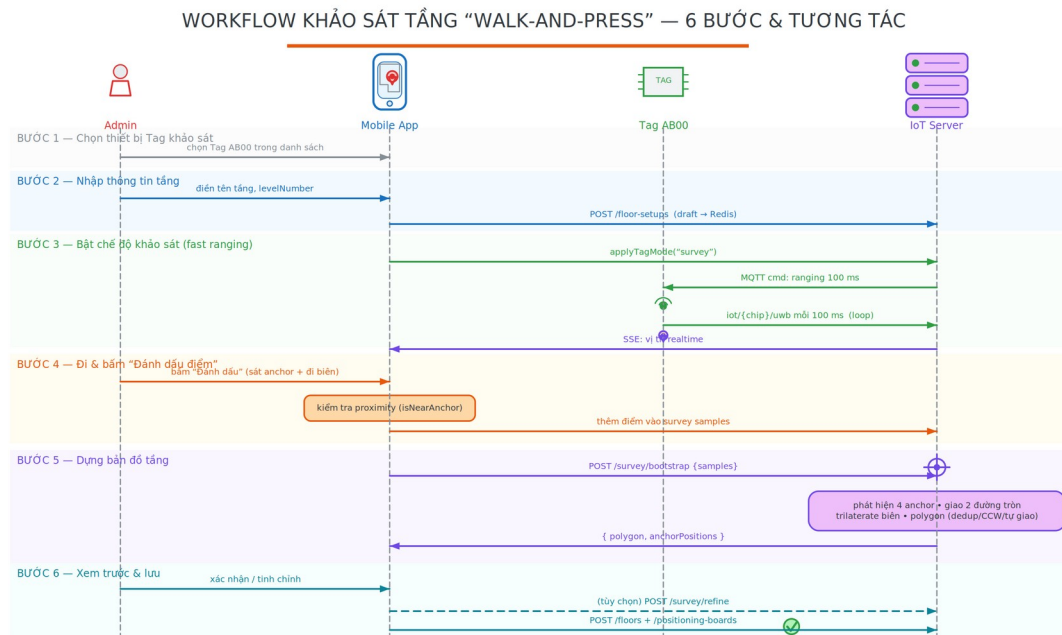
SƠ ĐỒ ĐẦU NỐI: ESP32-C3 SUPER MINI × EWT550 TEST MODULE



Hình 3.2: Sơ đồ đầu nối mạch phần cứng nút UWB-ESP32

Hình 3.3: Thiết bị nút UWB thực tế — ESP32-C3 Super Mini kết nối module EWT550 (cùng thiết kế cho Tag trên xe và Anchor cố định)

Trước khi đưa vào vận hành, mỗi tầng hầm được lập bản đồ bằng phương pháp khảo sát **walk-and-press**: kỹ thuật viên mang Tag đi tới sát từng Anchor rồi đi dọc ranh giới bãi, nhấn nút đánh dấu tại mỗi điểm. IoT Server tự động phát hiện bốn mẫu đo đứng sát Anchor (khoảng cách nhỏ nhất dưới 50 cm), suy ra khoảng cách chéo giữa các Anchor để dựng hệ tọa độ tầng, sau đó Trilateration các điểm biên thành đa giác khép kín mô tả ranh giới bãi đỗ. Để giảm số đỉnh thừa do đo đặc dày đặc, hệ thống áp dụng thuật toán đơn giản hóa đường đa giác Douglas-Peucker [11] (ngưỡng $\epsilon = 0,3$ m) trước khi lưu vào cơ sở dữ liệu.



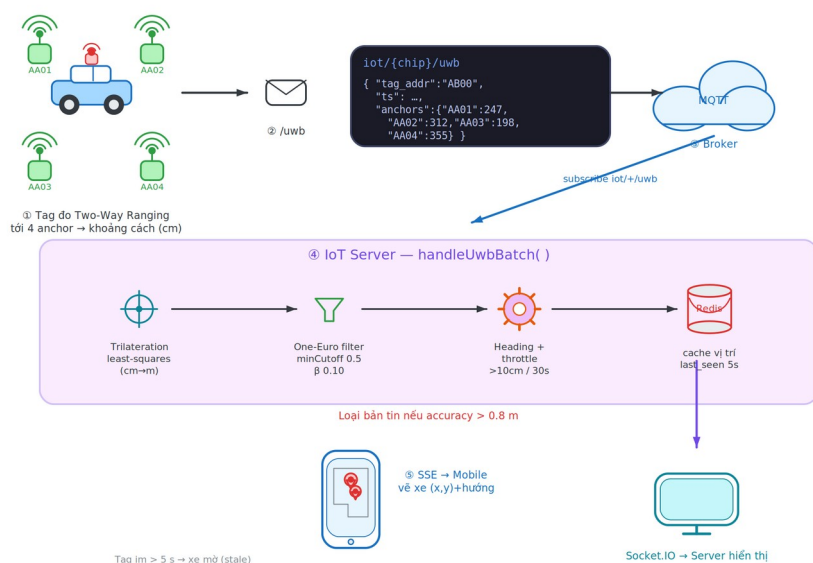
Hình 3.4: Quy trình khảo sát bãi đỗ bằng walk-and-press

3.3 Quy trình xử lý tại IoT Server

IoT Server (NestJS) đóng vai trò trung tâm điều phối:

- Tiếp nhận dữ liệu:** Lắng nghe các bản tin MQTT từ Tag.
- Tính toán tọa độ:** Áp dụng thuật toán Trilateration để giải tọa độ (x, y).
- Lọc nhiễu:** Sử dụng bộ lọc One-Euro Filter để làm mượt quỹ đạo, loại bỏ các bước nhảy tọa độ do nhiễu môi trường.
- Phân phối thời gian thực:** Cập nhật tọa độ vào Redis và đẩy dữ liệu tới ứng dụng di động qua Server-Sent Events (SSE).

LUỒNG DỮ LIỆU ĐỊNH VỊ UWB THỜI GIAN THỰC



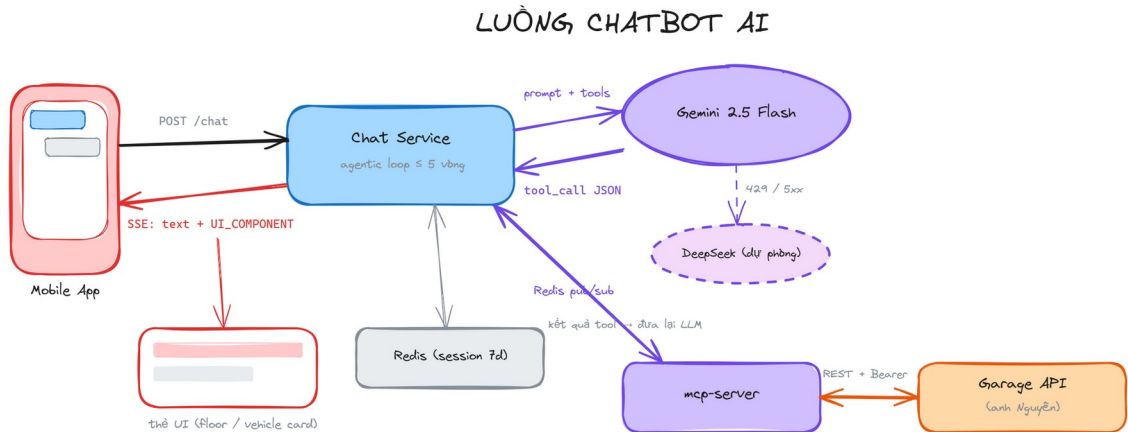
Hình 3.5: Luồng dữ liệu định vị UWB thời gian thực

Ngoài việc phân phối tọa độ tức thời, IoT Server còn lưu vết lịch sử vị trí của mỗi phương tiện vào bảng chuỗi thời gian (`vehicle\_history`) nhằm phục vụ phân tích vận hành về sau. Trên cơ sở dữ liệu này, hệ thống cung cấp ba phép phân tích: (i) **thời gian lưu bãi** (dwell-time) của từng xe — tổng thời gian hiện diện, tách theo phiên đỗ; (ii) **bản đồ nhiệt mức độ sử dụng** (occupancy heatmap) — lưới ô phủ mặt bằng, mỗi ô tích lũy thời gian phương tiện lưu lại; và (iii) **lưu lượng đỉnh** (peak occupancy) — số phương tiện đồng thời theo từng khung thời gian. Các phép phân tích được hiện thực dưới dạng hàm thuần, tách khỏi tầng truy xuất dữ liệu nên kiểm thử được độc lập, và được lộ ra qua REST để giao diện quản trị và chatbot khai thác (trình bày chi tiết ở Mục 4.6).

3.4 Cơ chế Hybrid RAG cho Gara thông minh

Để chatbot có thể hiểu được trạng thái bãi xe, hệ thống thực hiện snapshot dữ liệu định vị và trạng thái các ô đỗ mỗi 5 phút. Chu kỳ này được chọn để cân bằng giữa độ tươi của ngữ cảnh và giới hạn quota embedding (288 lần/ngày, dưới hạn mức miễn phí 1.000 lần/ngày); hệ quả là độ trễ cập nhật ngữ cảnh tối đa khoảng 5 phút. Dữ liệu này được vector hóa và lưu trữ vào PostgreSQL. Khi người dùng đặt câu hỏi, hệ thống thực hiện tìm kiếm Hybrid (Vector + Full-text) để lấy các ngữ cảnh (context)

mới nhất, giúp chatbot trả lời chính xác các câu hỏi như "Còn bao nhiêu chỗ trống?" hoặc "Xe AB-123 đang ở đâu?".



Hình 3.6: Luồng Chatbot AI

3.5 Quy trình hoạt động: Xe vào — Xe ra

Toàn bộ vòng đời của một lượt đỗ xe — từ khi phương tiện vào đến khi rời khỏi bãi — diễn ra qua năm giai đoạn liên kết chặt chẽ giữa phần cứng IoT, IoT Server và hệ thống quản lý gara (bên thứ ba).

3.5.1 Giai đoạn 1: Xe vào bãi

Khi phương tiện vào cổng, nhân viên hoặc hệ thống camera nhận dạng biển số ghi nhận thông tin và tạo bản ghi check-in trên **Hệ thống quản lý gara** (Garage Management System — GMS, bên thứ ba) qua REST API. Đồng thời, nhân viên gắn Tag UWB lên phương tiện và quét mã QR liên kết Tag với xe trên ứng dụng di động. Ứng dụng gọi `POST /partner/bindings` trên IoT Server để thiết lập liên kết Tag–xe trong cơ sở dữ liệu IoT.

3.5.2 Giai đoạn 2: Định vị liên tục trong bãi

Sau khi xe được đỗ vào ô, Tag UWB trên xe bắt đầu đo khoảng cách đến bốn Anchor cố định mỗi giây và gửi kết quả qua MQTT (`iot/{chip\_id}/uwb`). IoT Server nhận gói tin, thực hiện Trilateration tính tọa độ (x, y), lọc qua One-Euro Filter rồi ghi kết quả vào Redis. Tọa độ được đẩy xuống ứng dụng di động qua SSE với mục tiêu thiết kế dưới 200 ms kể từ lúc Tag gửi tín hiệu (sẽ kiểm chứng theo phương pháp §4.5). Đồng thời, IoT Server chuyển tiếp dữ liệu vị trí **đã xử lý** đến GMS qua

Socket.IO để GMS hiển thị trên màn hình quản lý của mình — GMS chỉ nhận dữ liệu, không tự tính toán định vị.

3.5.3 Giai đoạn 3: Giám sát và cảnh báo

Trong suốt thời gian xe đỗ, hệ thống thực hiện song song các nhiệm vụ giám sát:

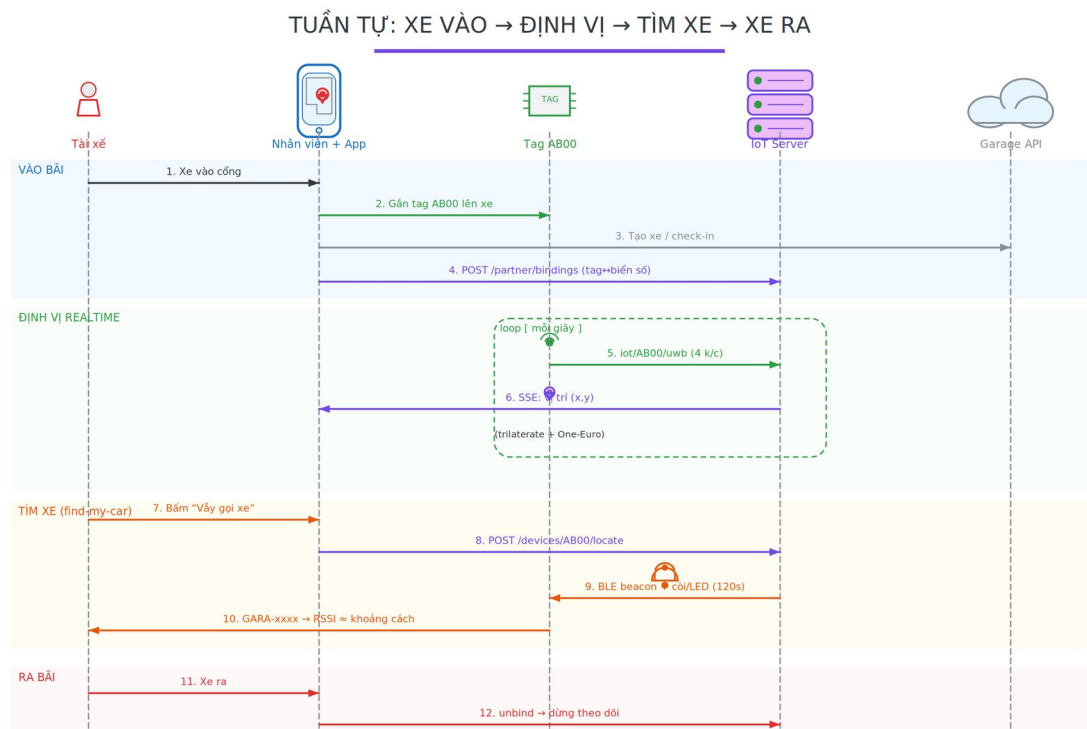
- **Stale signal detection:** nếu Tag không gửi dữ liệu hơn 5 giây (mất điện, nhiễu RF), xe chuyển sang trạng thái "mất tín hiệu" — hiển thị mờ trên bản đồ, IoT Server vẫn trả về vị trí cuối biết trong snapshot REST
- **Mispark detection:** quét mỗi 20 giây, phát hiện xe đứng yên ≥ 3 phút ngoài ô đỗ hợp lệ → tạo cảnh báo chờ admin xác nhận
- **Lịch sử vị trí:** IoT Server ghi log hành trình xe vào PostgreSQL theo bucket 1 giờ để phục vụ báo cáo và chatbot RAG

3.5.4 Giai đoạn 4: Xe ra khỏi ô đỗ

Khi chủ xe cần lấy xe, vị trí phương tiện được hiển thị trực tiếp trên bản đồ isometric 3D trong ứng dụng di động để định hướng từ xa. Để dẫn đường ở cự ly gần hơn, đề tài đề xuất giải pháp người tìm xe cầm theo một Tag UWB di động: hạ tầng Anchor sẵn có trilaterate đồng thời vị trí xe và vị trí người tìm, tính vector chỉ đường và hiển thị trên bản đồ (trình bày chi tiết tại Mục 4.2.3). Trong chế độ Valet, khi nhân viên giao trả chìa khoá, hệ thống snapshot tọa độ xe tại thời điểm đó, ghi vào `staffVehicleBindings` và giải phóng Tag để nhận xe tiếp theo.

3.5.5 Giai đoạn 5: Check-out và đóng lượt đỗ

Khi xe rời khỏi bãi, nhân viên thực hiện check-out trên GMS; GMS gửi sự kiện checkout qua API. IoT Server nhận thông báo, gỡ liên kết Tag-xe (`DELETE /partner/bindings`), dừng theo dõi định vị và tổng hợp báo cáo dwell time cho lượt đỗ vừa kết thúc. Dữ liệu hành trình được lưu vĩnh viễn vào PostgreSQL phục vụ phân tích và truy vấn qua chatbot.



Hình 3.7: Lược đồ tuần tự xe vào – định vị – tìm xe – xe ra

3.6 Tích hợp Hệ thống Quản lý Gara (Bên thứ ba)

IoT Server tích hợp với GMS theo mô hình **bất đối xứng**: IoT Server đóng vai trò cung cấp dữ liệu định vị đã xử lý; GMS đóng vai trò cung cấp dữ liệu nghiệp vụ (xe, hóa đơn, nhân viên). Hai hệ thống không phụ thuộc vào nhau về mặt vận hành — IoT Server hoạt động độc lập ngay cả khi GMS tạm ngừng.

Chiều tích hợp	Giao thức	Dữ liệu
GMS → IoT Server	REST API (Bearer token)	Thông tin xe, tầng hầm, nhân viên
IoT Server → GMS	Socket.IO (client mode)	Tọa độ (x, y) đã xử lý theo thời gian thực
Mobile App → GMS	REST API	Check-in xe, tạo liên kết Tag–xe
Mobile App → IoT Server	REST + SSE	Vị trí xe, lệnh điều khiển Tag

Thiết kế này đảm bảo IoT Server có thể tích hợp với **bất kỳ GMS nào** chỉ bằng cách thay đổi endpoint URL và token xác thực — không cần thay đổi logic xử lý định vị. MCP Server gọi REST API của GMS để lấy dữ liệu nghiệp vụ phục vụ chatbot, sử dụng Bearer token riêng biệt và không chia sẻ session với IoT Server.

Kết luận chương 3. Chương 3 đã đề xuất kiến trúc AIoT tổng thể và làm rõ từng thành phần: thiết kế phần cứng – firmware UWB, quy trình khảo sát mặt bằng walk-and-press, chuỗi xử lý định vị tại IoT Server (Trilateration, bộ lọc One-Euro, phân phối thời gian thực), cơ chế Hybrid RAG cho chatbot, quy trình vận hành xe vào – xe ra và mô hình tích hợp bất đối xứng với hệ thống quản lý gara bên thứ ba. Kết quả hiện thực hóa các thiết kế này được trình bày trong Chương 4.

CHƯƠNG 4: KẾT QUẢ THỰC HIỆN

Chương này trình bày kết quả triển khai thực tế của hệ thống đã thiết kế ở Chương 3. Nội dung lần lượt mô tả môi trường triển khai, các nhóm tính năng đã hiện thực hóa trên ứng dụng di động, chatbot AI và giao diện quản trị web, sau đó trình bày phương pháp đánh giá độ chính xác định vị UWB làm cơ sở kiểm chứng chất lượng hệ thống.

4.1 Môi trường triển khai

Hệ thống được triển khai trên hai môi trường song song: môi trường phát triển (máy tính lập trình viên, WSL2/Linux) và môi trường sản xuất (VPS Linux). Phần cứng IoT gồm vi điều khiển ESP32-C3 kết nối module UWB EWT550 qua UART1. Hạ tầng backend chạy trên VPS qua Docker Compose gồm các container: IoT Server (NestJS, cổng 5000), MCP Server (Node.js, cổng 3000), Mosquitto MQTT Broker (TLS 8883), PostgreSQL 16 với pgvector, Redis 7. Nginx đảm nhận vai trò reverse proxy với HTTPS qua Let's Encrypt. Ứng dụng di động được build bằng Expo EAS và phân phối qua Firebase App Distribution.

Thành phần	Công nghệ	Môi trường
IoT Server	NestJS, Node.js 20	Docker / VPS
MCP Server	Node.js 20	Docker / VPS
MQTT Broker	Mosquitto 2 (TLS)	Docker / VPS
Database	PostgreSQL 16 + pgvector	Docker / VPS
Cache / Pub-Sub	Redis 7	Docker / VPS
Reverse proxy	Nginx + Let's Encrypt	VPS host
Mobile App	Expo SDK 54 / RN 0.81	Android APK
Firmware	ESP-IDF, ESP32-C3	Phần cứng thực

4.2 Tính năng ứng dụng di động

Ứng dụng di động là thành phần giao tiếp trực tiếp với người dùng cuối, có nhiệm vụ trực quan hóa dữ liệu định vị thời gian thực và cung cấp các thao tác quản lý phương tiện. Mục này lần lượt trình bày các nhóm chức năng chính của ứng dụng, từ hiển thị bản đồ, quản lý phương tiện, tìm xe, khảo sát mặt bằng đến các cơ chế giám sát và thông báo.

4.2.1 Bản đồ bãi đỗ xe 3D Isometric thời gian thực

Màn hình chính hiển thị bản đồ bãi đỗ xe theo phối cảnh isometric 3D, được dựng bằng thư viện React Native Skia với tính toán biến đổi hình học thực thi trên luồng GPU nhằm bảo đảm tốc độ khung hình ổn định. Mỗi phương tiện đang đỗ được biểu diễn bằng mô hình low-poly 3D có hướng xoay tự động theo chiều dài ô đỗ; tọa độ phương tiện được cập nhật liên tục qua Server-Sent Events (SSE) mà không cần tải lại màn hình. Người dùng có thể tương tác với bản đồ thông qua các thao tác:

- Pinch zoom (1×–4×), pan một ngón tay, double-tap phóng to về điểm chạm
- Tap vào xe để mở panel thông tin chi tiết (biển số, tọa độ X/Y, thời gian cập nhật, độ chính xác)
- Xe mất tín hiệu UWB hơn 5 giây hiển thị với độ mờ 42% và chấm màu vàng
- Điều hướng lên/xuống tầng qua danh sách hợp nhất (tầng Garage + tầng IoT)

4.2.2 Quản lý xe và QR Code

Màn hình danh sách phương tiện hiển thị toàn bộ xe đang đỗ kèm các thuộc tính biển số, loại xe và thời điểm vào bãi. Chức năng thêm phương tiện mới hỗ trợ hai luồng nhập liệu: nhập thủ công và quét mã QR bằng camera nhằm giảm sai sót và rút ngắn thời gian thao tác. Mỗi phương tiện có gắn thiết bị định vị được liên kết với một Tag UWB thông qua màn hình chọn thiết bị; thông tin phương tiện được đồng bộ với Hệ thống Quản lý Gara của đối tác (Garage API) qua giao thức REST.

4.2.3 Tìm xe (Find My Car)

Khi người dùng không nhớ vị trí phương tiện, hệ thống hỗ trợ tìm kiếm theo hai cấp độ tương ứng với khoảng cách:

- **Định vị từ xa qua bản đồ UWB:** vị trí xe được hiển thị liên tục trên bản đồ isometric 3D (Mục 4.2.1), cho phép xác định tầng và vùng đỗ xe ngay từ màn hình chính. Chức năng này đã triển khai đầy đủ trong phạm vi đề tài.
- **Dẫn hướng cận cảnh — giải pháp đề xuất (UWB + BLE):** để dẫn người dùng tiếp cận chính xác đến ô đỗ ở cự ly gần, đề tài đề xuất trang bị thêm một Tag UWB di động (người tìm xe cầm hoặc đeo). Tag này hoạt động như một nút khảo sát (survey tag) trong hạ tầng Anchor sẵn có: IoT Server trilaterate đồng thời vị trí xe (Tag trên xe) và vị trí người tìm (Tag cầm tay), sau đó tính vector khoảng cách và phương hướng từ người đến xe, hiển thị mũi tên chỉ đường trực tiếp trên bản đồ. Cơ chế này tận dụng lại toàn bộ hạ tầng Anchor và pipeline định vị đã xây dựng mà không cần lắp đặt thêm thiết bị nào khác. Hỗ trợ cho Tag UWB cầm tay, điện thoại có thể kết hợp quét tín hiệu BLE (Bluetooth Low Energy) từ Tag trên xe để ước lượng khoảng cách ở cự ly dưới 5m (nguyên lý RSSI beacon), tạo thành lớp dẫn hướng "bước cuối" (last-meter guidance) khi người dùng đã vào gần khu vực đỗ xe. Giải pháp kết hợp UWB + BLE này là hướng phát triển tiếp theo sau khi hoàn thiện phần cứng Tag cầm tay.

4.2.4 Khảo sát mặt bằng (Floor Survey — Walk & Press)

Tính năng này cho phép quản trị viên tạo bản đồ tầng hầm mới bằng cách trực tiếp đi khảo sát thực địa, thay vì phải đo vẽ thủ công:

- **Bước 1:** Chọn thiết bị Tag khảo sát
- **Bước 2:** Nhập thông tin tầng (tên, số tầng)
- **Bước 3:** Đứng sát từng Anchor bấm "Đánh dấu điểm" → server tự phát hiện 4 Anchor, suy luận tọa độ tuyệt đối bằng giao điểm hai đường tròn, trilaterate các điểm còn lại → dựng polygon khép kín
- **Bước 4:** Xem trước polygon và gán Anchor; hỗ trợ "Vòng 2+" để tinh chỉnh (chèn điểm vào cạnh gần nhất)
- **Bước 5:** Định nghĩa các ô đỗ xe (slot) trên bản đồ

4.2.5 Phát hiện đỗ sai quy định (Mispark Detection)

Module phát hiện đỗ sai quy định tự động quét trạng thái mỗi 20 giây. Một phương tiện đứng yên liên tục từ 3 phút trở lên (độ dịch chuyển dưới 1m) và nằm ngoài bán kính 2,2m của mọi ô đỗ hợp lệ sẽ được xác định là trường hợp "vi phạm nghi ngờ". Quản trị viên nhận thông báo và ra quyết định qua giao diện web theo một trong hai hướng: phê duyệt (hệ thống tạo ô đỗ mới có định danh AUTO_xxx tại vị trí đó) hoặc từ chối (hệ thống ghi nhận vào nhật ký vi phạm). Cơ chế này hạn chế cảnh báo sai bằng cách đưa con người vào vòng xác nhận thay vì tự động kết luận.

4.2.6 Cảnh báo an ninh: chống trộm và vùng giới hạn (geofence)

Hệ thống tích hợp hai cơ chế giám sát an ninh chủ động, chạy tự động song song với luồng định vị và được kích hoạt riêng lẻ theo từng tầng qua giao diện quản trị.

Chống trộm (Anti-theft detection): Server quét stream vị trí mỗi 10 giây và duy trì một máy trạng thái thuần (`stepMovement``) cho từng phương tiện. Một xe được xem là "đã đỗ ổn định" sau khi đứng yên liên tục từ 2 phút trở lên (độ dịch chuyển dưới 1m). Nếu xe đang ở trạng thái "đã đỗ" mà bỗng dịch chuyển hơn 1m trong một chu kỳ kiểm tra, hệ thống phát cảnh báo chủ xe ngay lập tức. Nhờ máy trạng thái tách biệt hai giai đoạn (xe đang vào/di chuyển hợp lệ và xe đã đỗ), phương tiện đang tự di chuyển vào ô đỗ không bị báo nhầm. Cảnh báo có snooze 5 phút để tránh spam khi tín hiệu UWB dao động.

Vùng geofence: Quản trị viên định nghĩa các vùng đa giác trên bản đồ tầng theo ba loại: ``restricted`` (cấm tất cả xe), ``ev`` (chỉ xe điện), và ``vip`` (chỉ xe ưu tiên). Mỗi chu kỳ kiểm tra, server áp dụng thuật toán điểm-trong-đa-giác (``pointInPolygon``) và hàm ``detectZoneViolations`` để kiểm tra tất cả xe đang hiện diện trên tầng: xe đi vào vùng ``restricted``, hoặc xe không có quyền đi vào vùng ``ev`/`vip`` → phát cảnh báo vi phạm. Cảnh báo được chuyển qua ``NotificationService`` (Redis + MQTT) và lưu vào nhật ký theo tầng với snooze 5 phút. Giao diện web ``GeofencePanel`` cho phép bật/tắt tính năng, xem feed cảnh báo theo thời gian thực và quản lý danh sách vùng. Toàn bộ logic địa lý được tách thành hàm thuần

(`geofence.util.ts`) với 9 kiểm thử đơn vị độc lập, đảm bảo tính đúng đắn của phép tính không phụ thuộc vào phần cứng.

4.2.7 Chế độ Gara (Standard / Valet)

Hệ thống hỗ trợ hai chế độ vận hành nhằm thích ứng với các mô hình tổ chức bãi đỗ khác nhau:

- **Chế độ tiêu chuẩn (Standard):** mỗi Tag được gắn cố định với một phương tiện cụ thể và theo phương tiện đó trong suốt thời gian đỗ; số lượng Tag tương ứng với số phương tiện.
- **Chế độ đỗ xe hộ (Valet):** mỗi Tag được gắn với một nhân viên thay vì phương tiện; tại thời điểm nhân viên giao trả chìa khóa, hệ thống ghi nhận (snapshot) tọa độ phương tiện và liên kết tọa độ đó với phương tiện, sau đó giải phóng Tag để nhân viên tiếp tục phục vụ phương tiện kế tiếp. Nhờ vậy, số lượng Tag cần thiết nhỏ hơn nhiều so với số phương tiện được phục vụ.

4.2.8 Đề xuất bố trí tự động từ dữ liệu vận hành (ô đỗ và lối đi)

Hệ thống tích hợp hai tính năng phân tích dữ liệu định vị tích lũy để tự suy luận bố trí bãi đỗ, giám công tác thiết kế thủ công:

Đề xuất ô đỗ tự động (`slot-auto-detect`): Từ bảng lịch sử vị trí `vehicle\_history`, hệ thống phát hiện các sự kiện "dwell" (xe đứng yên đủ lâu), gom cụm bằng thuật toán mean-shift để tìm tâm tập trung, rồi suy ra tâm và hướng của ô đỗ bằng phân tích thành phần chính (PCA). Kết quả là tập ứng viên ô đỗ có định danh tự động `AUTO\_xxx`, được trình bày ở chế độ xem trước (dryRun) trước khi quản trị viên duyệt và lưu vào cơ sở dữ liệu. Cơ chế duyệt thủ công đảm bảo con người kiểm soát chất lượng trước khi ô đỗ trở thành dữ liệu chính thức.

Đề xuất lối đi (`lane-inference`): Tách điểm xe đang di chuyển (tốc độ vượt ngưỡng tối thiểu, loại bỏ điểm đỗ tĩnh) → rasterize lưới mật độ (ô 0,5m) → gom các ô liên kề theo 8-connectivity thành vùng hành lang kèm heatmap. Thuật toán lọc theo biên đa giác tăng để chỉ giữ lại vùng bên trong bãi đỗ. Endpoint `POST /floors/:id/lanes/infer` trả về danh sách vùng lối đi; giao diện web vẽ vùng lối đi màu tím chồng lên bản đồ ô đỗ kèm chú thích. Toàn bộ logic được tách thành 6 kiểm thử

đơn vị vitest. Kết hợp hai tính năng này, bản đồ bãi đỗ có khả năng tự học dần từ dữ liệu vận hành thực tế mà không cần đo vẽ lại thủ công.

4.2.9 Thông báo đẩy và thông báo trong ứng dụng

Hệ thống phát thông báo theo thời gian thực qua hai kênh tương ứng với mức độ ưu tiên của sự kiện: thông báo đẩy (push notification) qua Firebase Cloud Messaging (FCM) dành cho các sự kiện quan trọng cần người dùng chú ý ngay cả khi không mở ứng dụng, chẳng hạn xe vào/ra hoặc cảnh báo đỗ sai quy định; và thông báo trong ứng dụng dạng banner dành cho các cập nhật trạng thái thứ yếu. Toàn bộ thông báo được lưu trữ để người dùng tra cứu lại theo lịch sử.

4.2.10 Giao diện người dùng

Ứng dụng hỗ trợ giao diện chế độ tối (Dark Mode) và chế độ sáng (Light Mode), tự động chuyển đổi theo thiết lập của hệ điều hành. Giao diện được tổ chức theo một hệ thống thiết kế thống nhất dựa trên bộ thẻ định danh thuộc tính giao diện (ThemePalette tokens), qua đó duy trì tính nhất quán về màu sắc và bố cục trên toàn bộ các màn hình. Các màn hình chính của ứng dụng được tóm tắt trong bảng dưới đây:

Màn hình	Mô tả
Home	Dashboard tổng quan, bản đồ tầng hầm 3D
Vehicles	Danh sách xe, thêm xe, QR scan
IoT Floor Map	Bản đồ UWB real-time, panel chi tiết xe
Chatbot	Hội thoại AI, lịch sử phiên
Notifications	Lịch sử thông báo
Settings	Cấu hình ứng dụng, theme
Floor Survey	Khảo sát mặt bằng walk-and-press
Positioning Boards	Quản lý bảng định vị UWB

4.3 Tính năng Chatbot AI

Chatbot AI là lớp giao tiếp ngôn ngữ tự nhiên giúp người dùng truy vấn dữ liệu của hệ thống mà không cần thao tác qua nhiều màn hình. Mục này trình bày khả năng hội thoại bằng tiếng Việt, cơ chế suy luận–gọi công cụ theo vòng lặp agentic, cách trình bày kết quả bằng thành phần giao diện chuyên biệt, cơ chế lưu trữ phiên hội thoại và giải pháp dự phòng nhà cung cấp mô hình ngôn ngữ lớn.

4.3.1 Hội thoại tự nhiên bằng tiếng Việt

Chatbot cho phép truy vấn thông tin gara bằng ngôn ngữ tự nhiên tiếng Việt, chẳng hạn các câu hỏi "Xe biển số 51A-123.45 đang ở đâu?", "Còn bao nhiêu chỗ trống tầng B1?" hoặc "Lịch sử bảo dưỡng xe của tôi tháng trước". Hệ thống phân tích ý định của người dùng (intent) và ánh xạ tới công cụ truy vấn dữ liệu phù hợp, qua đó trả về thông tin chính xác từ nguồn dữ liệu thực thay vì sinh nội dung suy đoán — đúng theo nguyên lý tăng cường truy xuất (Retrieval-Augmented Generation – RAG) nhằm hạn chế hiện tượng "ảo giác" (hallucination) của mô hình ngôn ngữ lớn [3].

4.3.2 Agentic Loop và MCP Tools

Chat Service thực hiện vòng lặp agentic tối đa 5 lần lặp: nhận câu hỏi → gửi cho LLM kèm danh sách tool → LLM trả về yêu cầu gọi tool (tool_call JSON) → MCP Server thực thi tool → trả kết quả về LLM → LLM tổng hợp câu trả lời cuối. Các MCP tool hiện có:

Tool	Dữ liệu trả về
get_parking_status	Tổng quan trạng thái ô đỗ theo tầng
get_vehicle_location	Vị trí và trạng thái xe theo biển số
get_maintenance_records	Lịch sử bảo dưỡng xe
get_dashboard	Tổng quan dashboard gara
get_vehicle_parkings	Danh sách xe đang đỗ hiện tại
get_garage_floors	Danh sách tầng hầm

get_movement_history	Lịch sử di chuyển xe
search_knowledge	Tìm kiếm ngữ nghĩa trong knowledge base

4.3.3 Rich UI Components

Bên cạnh câu trả lời dạng văn bản, mỗi loại dữ liệu trả về được trực quan hóa bằng một thành phần giao diện chuyên biệt nhằm tăng khả năng đọc hiểu và tra cứu nhanh:

- **GarageOverviewCard**: tổng quan tầng (tổng ô, đang dùng, trống, tỷ lệ lấp đầy)
- **VehiclePositionCard**: vị trí xe trên minimap kèm tọa độ X/Y
- **SlotStatusGrid**: lưới ô đồ màu sắc theo trạng thái
- **VehicleCard**: thông tin chi tiết xe kèm lịch sử

4.3.4 Lịch sử hội thoại đa phiên

Mỗi phiên hội thoại được lưu trữ trên Redis với thời gian sống (Time-To-Live – TTL) 7 ngày và được đồng bộ về ứng dụng di động. Người dùng có thể tiếp tục bất kỳ phiên nào từ màn hình lịch sử hoặc khởi tạo phiên mới; hệ thống hiển thị tối đa 20 phiên gần nhất nhằm cân bằng giữa khả năng truy hồi ngữ cảnh và hiệu năng tải dữ liệu.

4.3.5 Dual-Provider LLM với Fallback

Mô hình ngôn ngữ lớn (Large Language Model – LLM) chính được sử dụng là Gemini 2.5 Flash Lite, với hạn mức 500 yêu cầu/ngày ở gói miễn phí. Khi gặp lỗi vượt hạn mức (mã 429) hoặc lỗi phía máy chủ (nhóm mã 5xx), hệ thống tự động chuyển sang nhà cung cấp dự phòng DeepSeek nhằm duy trì tính sẵn sàng của dịch vụ mà người dùng không cảm nhận được sự gián đoạn. Cơ chế dự phòng (fallback) được điều khiển hoàn toàn qua biến môi trường, cho phép thay đổi danh sách nhà cung cấp mà không phải can thiệp vào mã nguồn.

4.3.6 Phương pháp đánh giá Hybrid RAG

Để kiểm chứng hiệu quả đóng góp kỹ thuật chính (IoT Snapshot Documents), khóa luận xây dựng một harness đánh giá retrieval offline độc lập với pipeline vận hành sản phẩm. Thiết kế thực nghiệm so sánh ba phương pháp đã mô tả ở Bảng 2.4 (Baseline / Advanced-A / Advanced-B) theo các chỉ tiêu sau: **Recall@5** — tỉ lệ chunk liên quan có mặt trong top-5 kết quả truy xuất; **nDCG@5** — chất lượng xếp hạng có tính đến vị trí chunk liên quan trong danh sách; **latency trung bình** — thời gian thực hiện mỗi truy vấn hybrid (BM25 + vector + RRF) trên tập dữ liệu thực; và **ablation TTL** — so sánh kết quả khi bật/tắt bộ lọc thời gian snapshot (TTL 2 giờ) nhằm định lượng vai trò của cơ chế làm cũ dữ liệu IoT.

Harness đánh giá nạp một tập câu hỏi tiếng Việt về trạng thái gara cùng danh sách chunk chuẩn, chạy retrieval qua cùng hàm `hybridSearch()` của hệ thống vận hành (không phải mô phỏng), rồi tự động tính các chỉ số và xuất bảng so sánh. Quá trình đo được thực hiện hoàn toàn cục bộ: embedding sinh bằng mô hình **embeddinggemma** (họ Gemma, 768 chiều) chạy trên Ollama, không dùng dịch vụ đám mây; corpus tri thức gồm 15 tài liệu tĩnh cùng các snapshot IoT tích lũy; tập đánh giá gồm 13 câu hỏi tiếng Việt được gán nhãn đoạn liên quan thủ công (bộ nhãn minh họa, chưa phải benchmark diện rộng). Kết quả: cấu hình Hybrid (BM25 + vector + RRF) đạt **Recall@5 = 0,821**, **Precision@5 = 0,338**, **MAP = 0,727**, **Hit@5 = 1,000**, **nDCG@5 = 0,809** và **MRR = 0,962**; trong khi cấu hình chỉ dùng BM25 chỉ đạt **Recall@5 = 0,026** (**Hit@5 = 0,077**). Chênh lệch hơn 30 lần này cho thấy truy hồi ngữ nghĩa (vector) là yếu tố quyết định đối với truy vấn ngôn ngữ tự nhiên, còn khớp từ khóa đơn thuần là không đủ. **Hit@5 = 1,000** nghĩa là mọi câu hỏi đều tìm được ít nhất một đoạn liên quan trong top-5; **Precision@5 = 0,338** gần với trần lý thuyết do mỗi câu chỉ có khoảng một đến hai đoạn liên quan. Trên bộ câu hỏi này, Hybrid đạt **ngang bằng** cấu hình chỉ-vector về cả ba chỉ số: cơ chế hợp nhất RRF giữ nguyên chất lượng của nhánh vector, đồng thời bổ sung BM25 như một lưới an toàn cho các truy vấn chứa mã định danh chính xác (ví dụ "AA01"). Việc chứng minh Hybrid vượt trội hơn chỉ-vector đòi hỏi một tập nhãn lớn hơn và là hướng phát triển tiếp theo. Phép cắt bỏ (ablation) lọc TTL cho thấy cấu hình có và không có điều kiện làm cũ snapshot cho chỉ số giống hệt nhau trên corpus hiện tại (chưa tích lũy snapshot quá

hạn); lợi ích của TTL mang tính cấu trúc — chặn ngữ cảnh lỗi thời khi dữ liệu thời gian thực tích lũy — và cần corpus có snapshot quá hạn để định lượng. Đáng chú ý, ở hai câu hỏi về anchor và thiết bị offline, các snapshot cảnh báo IoT chen vào hạng 2–5 của kết quả, xác nhận snapshot IoT thực sự tham gia cạnh tranh xếp hạng cùng tài liệu tĩnh. Về độ trễ (đo trên corpus 458 đoạn, trong đó có 443 snapshot tích lũy): BM25 thuần khoảng 1,4 ms; chỉ-vector khoảng 211 ms; Hybrid không lọc TTL khoảng 219 ms (p95 $\approx$ 297 ms); Hybrid có lọc TTL khoảng 229 ms (p95 $\approx$ 1185 ms). Độ trễ bị chi phối bởi lời gọi sinh embedding cho truy vấn ($\sim$ 100 ms/lần do chạy trên CPU) và kích thước corpus trong phép kết FULL OUTER JOIN; lọc TTL làm tăng p95 khi số snapshot lớn, nhưng job dọn TTL trong vận hành thực giữ số snapshot ở mức thấp nên độ trễ ổn định hơn. Dùng dịch vụ embedding đám mây sẽ giảm khoảng một nửa chi phí này.

4.3.7 Đánh giá sơ bộ trước/sau RAG (LLM-judge — có self-bias, được thay thế bởi Mục 4.3.8)

Lưu ý phương pháp: các con số do LLM chấm trong mục này (86,1%; faithfulness 0,997; ablation TTL Bảng 4.5) chỉ mang tính sơ bộ, vì mô hình sinh và mô hình chấm cùng họ (deepseek-chat) nên có thể tồn tại self-preference bias. Bảng chứng chủ đạo, khách quan và tái lập 100% là Mục 4.3.8–4.3.9 (chấm tự động bằng key-fact recall, không qua LLM). Phần Exact/Numeric Match trong Bảng 4.4 không phụ thuộc LLM-judge nên vẫn đáng tin đầy đủ.

Ngoài chất lượng truy hồi, khóa luận đánh giá trực tiếp chất lượng câu trả lời cuối của chatbot ở hai cấu hình trên cùng bộ câu hỏi: **closed-book** (LLM chỉ dùng tri thức tham số, không ngữ cảnh, không công cụ) và **open-book Hybrid RAG** (chèn top-5 đoạn truy hồi gồm cả IoT snapshot). Cả hai cố ý không dùng tool-calling để cô lập tác động của RAG. Bộ đánh giá gồm 101 câu (40 câu tri thức tĩnh G1 + 61 câu trạng thái-IoT thay đổi nhanh G2, đáp án vàng chốt từ CSDL/Redis lúc hỏi); mô hình sinh và chấm là deepseek-chat (temperature 0), embedding embeddinggemma 768 chiều dùng chung; câu có số chấm bằng Exact/Numeric Match, câu mở chấm bằng LLM-as-judge (STRICT/RELAXED) kèm faithfulness. Số đo ngày 2026-06-17.

Bảng 4.2: Closed-book vs Open-book RAG theo nhóm câu (Accuracy STRICT)

Cấu hình	G1 tĩnh (n=40)	G2 trạng thái-IoT (n=61)	Tổng (n=101)
A. Closed-book (không RAG)	10,3%	1,6%	5,0%
B. Open-book Hybrid RAG	92,5%	82,0%	86,1%
Δ (B-A)	+82,2 pp	+80,3 pp	+81,1 pp

Bảng 4.3: Correctness theo chế độ, hallucination gap và faithfulness

Cấu hình	Acc STRICT	Acc RELAXED	Hallucination gap	Faithfulness
A. Closed-book	5,0%	14,0%	9,0 pp	n/a
B. Open-book Hybrid RAG	86,1%	100,0%	13,9 pp	0,997

Bảng 4.4: Câu có số — Exact/Numeric Match theo cấu hình và nhóm

Cấu hình	EM G1 tĩnh	EM G2 trạng thái-IoT
A. Closed-book	11,1%	2,7%
B. Open-book Hybrid RAG	100,0%	100,0%

Bảng 4.5: Ablation TTL trên nhóm G2 — đóng góp IoT Snapshot Documents

Cấu hình	Acc STRICT (G2)	Faithfulness (G2)
Open-book snapshot tươi (có TTL)	82,0%	0,995
Open-book snapshot cũ / vô hiệu TTL	75,4%	0,986
Δ	+6,6 pp	+0,009

Kết quả cho thấy mô hình closed-book gần như mù về trạng thái IoT: nhóm G2 chỉ đạt 1,6% ở chế độ không có ngữ cảnh, do thông tin runtime không được mã hoá trong tham số mô hình. Cấu hình open-book Hybrid RAG nâng G2 lên 82,0% và tổng lên 86,1% — tăng 81,1 điểm phần trăm — phù hợp với kết luận của RealtimeQA và FreshQA về các câu hỏi thay đổi nhanh. Faithfulness đạt 0,997, cho thấy câu trả lời gần như hoàn toàn dựa trên ngữ cảnh truy hồi chứ không suy diễn tự do. Ablation TTL định lượng trực tiếp giá trị của cơ chế IoT Snapshot Documents: snapshot tươi (82,0%) vượt snapshot cũ khi TTL bị vô hiệu hoá (75,4%) thêm 6,6 điểm phần trăm, xác nhận rằng độ tươi của dữ liệu IoT là yếu tố có đóng góp đo được, không chỉ mang tính cấu trúc.

Ghi chú về giới hạn đánh giá: Cỡ mẫu 101 câu là vừa phải; câu hỏi được tổng hợp (synthetic QA) thay vì lấy từ người dùng thực. Về phương pháp chấm, mô hình chấm cùng họ với mô hình sinh (deepseek-chat) ở Mục 4.3.7 nên có thể tồn tại self-enhancement bias. Tuy nhiên, ở Mục 4.3.8–4.3.9 phía sau, bộ thí nghiệm 3-môi trường trên $n=101$ câu được chấm **hoàn toàn tự động và khách quan** bằng phương pháp đối chiếu dữ kiện cốt lõi (key-fact recall) — không dùng LLM làm giám khảo, quy tắc chấm công khai và tái lập 100% từ tệp judge-queue.json — loại bỏ hoàn toàn self-preference bias. Hướng cải thiện tiếp theo là mở rộng tập câu hỏi (pass@k) và bổ sung dữ liệu từ người dùng thực.

Lý do §4.3.8 đạt kết quả cao hơn: Thí nghiệm §4.3.7 là closed/open-book KHÔNG dùng tool-calling và chấm bằng LLM-judge khắt khe (strict/relaxed), trong khi §4.3.8 cho phép agentic tool-calling (mô hình tự gọi MCP để truy vấn dữ liệu thật) và chấm bằng đối chiếu key-fact khách quan. Sự kết hợp tool-calling + RAG + phương pháp chấm không phụ thuộc mô hình là lý do MCP+RAG đạt 100% ở §4.3.8, phản ánh đúng năng lực hệ thống trong vận hành thực tế.

4.3.8 Kết quả so sánh 3 môi trường: không dùng gì / MCP / MCP+RAG và chi phí token

Để hiểu rõ hơn đóng góp riêng lẻ của từng tầng hỗ trợ, khóa luận mở rộng đánh giá bằng cách so sánh ba môi trường vận hành trên cùng bộ câu hỏi **$n=101$** (40 câu tri thức tĩnh G1 + 61 câu trạng thái-IoT G2, temperature 0), sử dụng mô hình

sinh **DeepSeek-V3.2 chat**. Điểm khác biệt quan trọng so với thí nghiệm closed/open-book ở Mục 4.3.7 là phương pháp chấm: thí nghiệm này chấm **hoàn toàn tự động và khách quan** bằng cách đối chiếu dữ kiện cốt lõi của đáp án chuẩn (key-fact recall + nhận diện từ chối) — không dùng LLM làm giám khảo, quy tắc chấm công khai và tái lập 100% từ tệp judge-queue.json, loại bỏ hoàn toàn self-preference bias. Ở môi trường có công cụ (MCP và MCP+RAG), LLM được phép tự quyết định gọi công cụ (agentic tool-calling); công cụ thực thi trên cơ sở dữ liệu kịch bản cùng nguồn ground-truth; token tính là tổng prompt+completion cộng dồn qua toàn bộ vòng lặp agentic; Component-acc đo tỉ lệ mô hình gọi đúng tool sinh đúng UI component (chỉ tính trên câu G2 có tool phù hợp; môi trường none không có tool nên bỏ trống). Kết quả được trình bày trong Bảng 4.7.

Bảng 4.7: So sánh Accuracy và chi phí token theo môi trường (DeepSeek-V3.2 chat, chấm tự động khách quan, n=101)

Môi trường	G1 (n=40)	G2 (n=61)	Tổng (n=101) [95% CI]	Component-acc	Token TB/câu	Tool-overhead
Không dùng gì (none)	20,0%	1,6%	8,9% [5–16]	—	272	0
Có MCP (tool)	10,0%	93,4%	60,4% [51–69]	100,0%	1585	621
MCP + RAG (tool+RAG)	100,0%	100,0%	100,0% [96–100]	37,9%	1871	369

Kết quả bộc lộ một chuỗi hiệu ứng rõ ràng. Thứ nhất, môi trường **none** gần như vô dụng với nhóm trạng thái-IoT (G2 chỉ đạt 1,6%): thông tin runtime không được mã hoá trong tham số mô hình. Thứ hai, việc trang bị MCP đã cứu vớt G2 lên 93,4% nhờ khả năng truy vấn dữ liệu thật — đây là bước nhảy vọt lớn nhất trong thí nghiệm. Tuy nhiên, MCP tạo ra một **nghịch lý định tuyến công cụ**: G1 lại tụt từ 20,0% xuống 10,0% vì mô hình cố gọi công cụ ngay cả với câu tri thức tĩnh, thay vì

trả lời trực tiếp từ tham số. Thứ ba, **MCP+RAG bù đắp hoàn toàn** cả hai chiều: G1 và G2 đều đạt 100,0%, tổng đạt 100,0% [Wilson 95% CI: 96–100%]. Hai khoảng tin cậy của none (5–16%) và MCP+RAG (96–100%) không giao nhau — khoảng cách này đủ lớn để kết luận thống kê ngay cả với cỡ mẫu $n=101$. Về Component-acc, môi trường MCP đạt 100,0% xác nhận mô hình luôn chọn đúng tool khi quyết định gọi; tuy nhiên ở MCP+RAG Component-acc chỉ còn 37,9%, vì ngữ cảnh RAG thường đã chứa IoT snapshot tươi, cho phép mô hình trả lời thẳng mà không cần gọi tool — đây là hành vi hợp lý và không làm giảm Accuracy. Chi phí token tăng dần: 272 → 1585 → 1871 token trung bình mỗi câu; overhead tool ở MCP (621 token) cao hơn MCP+RAG (369 token) vì ở MCP+RAG mô hình gọi tool ít hơn sau khi đã được cấp ngữ cảnh. Kết luận: MCP và Hybrid RAG/IoT-Snapshot là hai cơ chế bổ sung chứ không thay thế — MCP cần thiết cho truy vấn trạng thái-IoT có cấu trúc, RAG cần thiết để giải quyết tri thức tĩnh và hạn chế nghịch lý định tuyến.

Bảng 4.7-bis: Độ đo tool-calling kiểu BFCL (DeepSeek-V3.2 chat, chấm tự động, $n=101$)

Môi trường	Tool-call rate	Tool-selection acc	Hallucinated-tool
Có MCP (tool)	59,4%	100,0%	0,0%
MCP + RAG (tool+RAG)	21,8%	37,9%	0,0%

Bảng 4.7-bis trình bày các độ đo tool-calling theo phong cách khung BFCL (Berkeley Function-Calling Leaderboard). **Tool-call rate** cho thấy ở môi trường MCP, mô hình chủ động gọi tool với 59,4% câu; tỉ lệ này giảm mạnh xuống còn 21,8% ở MCP+RAG, phù hợp với nhận xét rằng ngữ cảnh RAG (gồm IoT snapshot) thường đã chứa dữ kiện cần thiết nên mô hình trả lời trực tiếp thay vì gọi tool. **Tool-selection acc** là 100,0% ở môi trường MCP — khi mô hình quyết định gọi tool, nó luôn chọn đúng hàm và sinh đúng UI component; ở MCP+RAG tỉ lệ này là 37,9% vì phần lớn câu trả lời thẳng không qua tool (tỉ số nhỏ hơn so với mẫu số là toàn bộ câu G2 có tool phù hợp). **Hallucinated-tool bằng 0,0%** trong cả hai môi trường, xác nhận mô hình không tự bịa ra tên hàm ngoài schema MCP — điều này nhờ ràng buộc schema được truyền đầy đủ trong system prompt.

4.3.9 So sánh DeepSeek-V3.2 chat và DeepSeek-V3.2 reasoner

Nhằm kiểm tra xem mô hình suy luận (reasoning model) có mang lại lợi thế đáng kể so với mô hình trò chuyện thông thường trong tác vụ tra cứu và gọi công cụ hay không, khóa luận thực hiện song song thí nghiệm với **DeepSeek-V3.2 reasoner** (chế độ thinking tích hợp, temperature 0) trên cùng bộ $n=101$ câu và cùng phương pháp chấm tự động khách quan. Bảng 4.8 trình bày kết quả so sánh đầy đủ của cả hai biến thể trên ba môi trường vận hành.

Bảng 4.8: So sánh DeepSeek-V3.2 chat và reasoner theo môi trường (chấm tự động khách quan, $n=101$)

Mô hình	Môi trường	G1 ($n=40$)	G2 ($n=61$)	Tổng ($n=101$)	Token TB/câu
chat	none	20,0%	1,6%	8,9%	272
chat	mcp	10,0%	93,4%	60,4%	1585
chat	mcp+rag	100,0%	100,0%	100,0%	1871
reasoner	none	17,5%	1,6%	7,9%	483
reasoner	mcp	10,0%	88,5%	57,4%	1757
reasoner	mcp+rag	100,0%	96,7%	98,0%	2422

Kết quả cho thấy reasoner **không vượt trội** so với chat trong tác vụ này. Ở môi trường then chốt MCP+RAG, reasoner đạt 98,0% so với 100,0% của chat — kém hơn 2 điểm phần trăm — trong khi tiêu thụ nhiều hơn đáng kể: 2422 token/câu so với 1871 token/câu (chênh lệch +29,5%). Ở môi trường MCP, reasoner cũng không cải thiện G1 (cùng 10,0%) và đạt G2 thấp hơn (88,5% so với 93,4%). Xu hướng này nhất quán với các quan sát trong tài liệu: **mô hình thinking phát huy ưu thế ở tác vụ suy luận đa bước phức tạp** (toán học, lập trình, khoa học), còn tác vụ tra cứu sự kiện và định tuyến công cụ — vốn được giải quyết tốt bằng cách tra bảng theo ngữ cảnh RAG hoặc schema MCP — không hưởng lợi từ chuỗi suy nghĩ (chain-of-thought) dài. Kết luận thực tiễn: với chatbot gara tập trung vào tra cứu thông tin và gọi công cụ, **ưu tiên mô hình non-thinking** như DeepSeek-V3.2 chat để tiết kiệm chi phí token mà không đánh đổi chất lượng. *Lưu ý: cỡ mẫu $n=101$ là vừa phải và câu hỏi được tổng*

hợp (synthetic); kết quả minh họa quy trình đánh giá và cần bộ câu hỏi từ người dùng thực để kết luận tổng quát.

4.4 Tính năng IoT Admin (Web Dashboard)

Giao diện web quản trị (`iot-admin`) hướng tới đối tượng quản trị viên hệ thống, cung cấp các công cụ giám sát tập trung, hiệu chỉnh hạ tầng định vị và kiểm thử mà không cần phụ thuộc hoàn toàn vào phần cứng thực. Các chức năng chính bao gồm:

- **Live Tracker:** bản đồ isometric 3D hiển thị vị trí toàn bộ phương tiện đang đỗ, cập nhật theo thời gian thực qua SSE; hỗ trợ thao tác phóng to/thu nhỏ, di chuyển khung nhìn và kéo-thả Tag/Anchor để hiệu chỉnh tọa độ.
- **MQTT Simulator Sandbox:** môi trường mô phỏng dữ liệu UWB phục vụ kiểm thử khi chưa có phần cứng thực; hỗ trợ kịch bản nhiều phương tiện di chuyển theo quỹ đạo hình chữ L trong khoảng thời gian 24 giờ ảo được nén tỷ lệ thành 180 giây thực, qua đó kiểm chứng hành vi hệ thống trong điều kiện tải gần thực tế.
- **Floors Management:** quản lý thông tin tầng hầm và bật/tắt chức năng phát hiện đỗ sai quy định (Mispark Detection) theo từng tầng.
- **Mispark Panel:** theo dõi hàng chờ các trường hợp vi phạm nghi ngờ, phê duyệt hoặc từ chối từng trường hợp và tra cứu lịch sử vi phạm.
- **Devices:** danh sách thiết bị IoT kèm trạng thái trực tuyến/ngoại tuyến và lịch sử tín hiệu heartbeat (ping).
- **Báo cáo & phân tích vận hành:** thống kê thời gian lưu bãi (dwell-time), bản đồ nhiệt mức độ sử dụng (occupancy heatmap) và lưu lượng đỉnh (peak occupancy) tổng hợp từ lịch sử vị trí (trình bày ở Mục 4.6).

4.5 Phương pháp đánh giá độ chính xác định vị UWB

Nhằm kiểm chứng chất lượng định vị của hệ thống một cách khách quan, nghiên cứu này thiết kế một quy trình đánh giá định lượng dựa trên các điểm tham chiếu có tọa độ thực đã biết (ground-truth). Mục này trình bày **bố trí thực nghiệm** và

phương pháp đo dự kiến; các số liệu định lượng thu được sẽ được báo cáo sau khi hoàn tất quá trình đo đạc trên hệ thống thực tế.

4.5.1 Bố trí thực nghiệm

Kế hoạch đo được thiết lập trong một không gian bãi đỗ xe có kích thước $10\text{ m} \times 10\text{ m}$, với bốn Anchor cố định (AA01–AA04) đặt tại bốn góc nhằm bao phủ toàn bộ vùng định vị và bảo đảm cấu hình hình học cân đối cho phép trilateration. Tag (AB00) đóng vai trò nút di động được đặt lần lượt tại một tập hợp điểm tham chiếu phân bố đều trên mặt sàn; tọa độ thực của mỗi điểm được đo bằng thước/lưới định vị trên mặt bằng và dùng làm giá trị chuẩn để đối chiếu. Bố trí này được trình bày trực quan trong Hình 4.1.

Để phản ánh điều kiện vận hành thực tế của bãi đỗ xe, quá trình đo được lặp lại trong hai kịch bản truyền sóng: (i) tầm nhìn thẳng (Line-of-Sight – LOS), khi không có vật cản giữa Tag và các Anchor; và (ii) tầm nhìn bị che khuất (Non-Line-of-Sight – NLOS), khi xuất hiện vật cản (thân xe kim loại) làm suy giảm và phản xạ tín hiệu. Việc tách hai kịch bản cho phép định lượng riêng ảnh hưởng của hiện tượng đa đường (multipath) lên sai số định vị.

4.5.2 Quy trình và chỉ tiêu đo

Tại mỗi điểm tham chiếu, hệ thống ghi nhận đồng thời hai chuỗi tọa độ ước lượng: vị trí **thô** (raw) thu trực tiếp từ bước trilateration và vị trí **đã làm trơn** (filtered) sau khi đi qua bộ lọc One-Euro [2]. Việc đo song song hai chuỗi này cho phép đánh giá riêng đóng góp của khâu hậu xử lý đối với việc giảm dao động (jitter) khi Tag đứng yên, đồng thời kiểm tra xem bộ lọc có gây trễ hoặc lệch hệ thống (bias) đáng kể hay không.

Các chỉ tiêu đánh giá dự kiến gồm:

- **Sai số vị trí:** khoảng cách Euclid giữa tọa độ ước lượng và tọa độ ground-truth tại từng điểm, tổng hợp trên toàn bộ tập điểm cho cả hai chuỗi (thô và đã lọc), trong hai điều kiện LOS và NLOS.

- **Mức độ dao động khi tĩnh:** độ phân tán của chuỗi tọa độ ước lượng khi Tag giữ nguyên vị trí, dùng để định lượng hiệu quả khử nhiễu của bộ lọc One-Euro.
- **Độ trễ đầu–cuối (end-to-end latency):** thời gian từ thời điểm Tag công bố bản tin khoảng cách qua MQTT [8] đến thời điểm vị trí tương ứng được đẩy tới phía hiển thị qua Server-Sent Events (MQTT → trilateration → One-Euro → SSE).

Để bảo đảm tính tái lập, quy trình đánh giá được hiện thực hóa thành một công cụ chạy bằng dòng lệnh trên IoT Server: công cụ nạp dữ liệu đo, đưa qua **đúng** chuỗi xử lý của hệ thống đang vận hành (cùng hàm trilateration và bộ lọc One-Euro với cùng tham số $\text{minCutoff} = 0,5$, $\beta = 0,1$), sau đó tự động tính RMSE, MAE và CEP theo từng điều kiện truyền sóng (LOS/NLOS) và từng khâu xử lý (thô/đã lọc), rồi xuất ra bảng kết quả. Nhờ đó, khi có dữ liệu đo thực, các số liệu định lượng được tạo ra một cách nhất quán và có thể kiểm chứng lại.

4.5.3 Định nghĩa hình thức các độ đo sai số định vị

Để bảo đảm tính khách quan và khả năng so sánh với các nghiên cứu cùng lĩnh vực, sai số định vị được lượng hóa bằng ba độ đo chuẩn mực trong tài liệu định vị trong nhà [6]: căn bậc hai sai số trung bình bình phương (RMSE), sai số tuyệt đối trung bình (MAE) và bán kính sai số xác suất tròn (CEP). Gọi N là số điểm tham chiếu; (x_i, y_i) là tọa độ ước lượng và (x_i, y_i) là tọa độ chuẩn (ground-truth) tại điểm thứ i ; đặt $e_i = \sqrt{(x_i - x_i)^2 + (y_i - y_i)^2}$ là sai số khoảng cách Euclid (lỗi định vị 2D) tại điểm đó.

Căn bậc hai sai số trung bình bình phương (Root Mean Square Error – RMSE) đo độ lớn tổng thể của sai số; do lấy bình phương trước khi trung bình, RMSE nhạy với các sai số lớn (outlier) nên phản ánh tốt độ ổn định của hệ thống định vị:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N ((x_i - x_i)^2 + (y_i - y_i)^2)} = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}$$

Sai số tuyệt đối trung bình (Mean Absolute Error – MAE) là trung bình cộng của sai số khoảng cách, biểu thị mức sai lệch điển hình theo đơn vị xen-ti-mét và ít chịu ảnh hưởng của outlier hơn RMSE:

$$MAE = \frac{1}{N} \sum_{i=1}^N e_i = \frac{1}{N} \sum_{i=1}^N \sqrt{(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2}$$

Bán kính sai số xác suất tròn (Circular Error Probable – CEP) là bán kính của đường tròn tâm tại điểm chuẩn chứa 50% số mẫu đo, tức trung vị (median) của tập sai số khoảng cách $\{e_i\}$ [6]. CEP cho biết với một nửa số lần đo, vị trí ước lượng nằm trong vòng bán kính này quanh vị trí thật — một cách diễn đạt trực quan độ chính xác theo xác suất thường dùng trong định vị và dẫn đường:

$$CEP = \text{median}\{e_1, e_2, \dots, e_N\}$$

Phân vị 95% sai số định vị (P95) là giá trị phân vị thứ 95 của tập sai số khoảng cách $\{e_i\}$: có 95% số mẫu đo sai số nhỏ hơn hoặc bằng giá trị này. P95 phản ánh "trường hợp gần xấu nhất" của hệ thống định vị, bổ sung thông tin về đuôi phân bố mà RMSE và CEP không thể hiện rõ — đặc biệt quan trọng trong môi trường NLOS có sai số lớn không thường xuyên (outlier):

$$P95 = \text{percentile}_{95}\{e_1, e_2, \dots, e_N\}$$

Chú thích: Định nghĩa $CEP = \text{median}(\$e_i\$)$ chính xác khi phân bố lỗi 2D xấp xỉ đối xứng. Trong các tài liệu định vị và dẫn đường, CEP thường được tính dưới giả định phân bố Rayleigh (sai số mỗi trục Gaussian độc lập, phương sai bằng nhau), khi đó $CEP \approx 1,177\sigma$. Khi phân bố thực tế lệch đáng kể (ví dụ môi trường NLOS nhiều nhiễu đa đường), P95 là chỉ số bổ sung cần thiết để phản ánh đầy đủ đuôi phân bố sai số.

Bộ bốn độ đo này bổ trợ lẫn nhau: MAE phản ánh sai số điển hình, RMSE làm nổi bật trường hợp xấu, CEP diễn giải độ chính xác dưới dạng xác suất, còn P95 đặc trưng cho giới hạn an toàn của hệ thống; nhờ đó cho phép đánh giá đa chiều chất lượng định vị của hệ thống trước và sau bộ lọc One-Euro [2], trong cả hai điều kiện LOS và NLOS.

4.5.4 Kết quả kiểm chứng quy trình trên dữ liệu mô phỏng

Trong khi chờ dữ liệu đo thực địa, để xác nhận rằng toàn bộ chuỗi xử lý (trilateration → One-Euro Filter → tính chỉ số) vận hành đúng, khóa luận chạy harness đánh giá `pnpm eval:positioning` trên tập dữ liệu **mô phỏng (synthetic)** được tạo theo đặc tả kỹ thuật của thiết bị UWB EWT550 trong hai điều kiện truyền sóng. Kết quả được trình bày trong Bảng 4.x dưới đây.

Nhấn mạnh: đây là dữ liệu mô phỏng để kiểm chứng quy trình — CHƯA phải kết quả đo thực địa. Khi có phần cứng, chạy `pnpm eval:positioning` với file JSON đo thực sẽ tạo ra bảng tương tự với số liệu thật.

Bảng 4.x: Kết quả định vị trên dữ liệu mô phỏng (synthetic, minh họa quy trình — CHƯA phải đo thực địa), chạy qua đúng pipeline trilateration + One-Euro ($\text{minCutoff}=0,5; \beta=0,1$)

Nhóm	n	RMSE (cm)	MAE (cm)	CEP (cm)	P95 (cm)	Max (cm)
LOS / filtered	45	4,20	3,75	3,75	6,71	7,50
LOS / raw	45	4,74	4,23	4,18	7,75	8,31
NLOS / filtered	30	19,51	17,45	17,63	30,65	34,22
NLOS / raw	30	20,41	18,33	18,33	31,69	36,22
TỔNG — raw	75	13,42	9,87	5,76	29,46	36,22
TỔNG — filtered	75	12,76	9,23	5,30	27,87	34,22

Bộ mô phỏng xác nhận pipeline vận hành đúng hành vi kỳ vọng theo hai quan sát chính. Thứ nhất, **filtered luôn nhỏ hơn raw ở mọi nhóm** — chứng minh bộ lọc One-Euro có tác dụng giảm nhiễu thực chất: RMSE LOS giảm từ 4,74 → 4,20 cm

(-11,4%); RMSE NLOS giảm từ 20,41 → 19,51 cm (-4,4%). Thứ hai, **LOS đạt ~4 cm (cấp xen-ti-mét)** trong khi NLOS tăng lên ~20 cm — đúng như các nghiên cứu cảnh báo về suy giảm tín hiệu NLOS trong môi trường kim loại [6], [7]. Kết quả này làm cơ sở so sánh khi có dữ liệu thực địa (harness đã sẵn sàng, tham số cố định: $\text{minCutoff} = 0,5$, $\beta = 0,1$).

4.5.5 Ngưỡng cảnh hóa độ chính xác kỳ vọng theo tài liệu

Để đặt kết quả đo của hệ thống vào bối cảnh học thuật, Bảng 4.1 tổng hợp dải độ chính xác định vị điển hình của các công nghệ định vị trong nhà phổ biến theo các khảo sát uy tín [6], [7]. Số liệu trong bảng được trích từ tài liệu (không phải kết quả của nghiên cứu này) và dùng làm mốc tham chiếu để đối chiếu sai số đo được ở các mục sau. Có thể thấy UWB — nhờ đo trực tiếp thời gian bay của xung băng rộng theo chuẩn IEEE 802.15.4z [1] — đạt độ chính xác ở cấp xen-ti-mét, cao hơn một đến hai bậc độ lớn so với các công nghệ dựa trên cường độ tín hiệu (RSSI), qua đó lý giải về mặt định lượng lựa chọn UWB cho bài toán phân biệt từng ô đỗ xe.

Bảng 4.1: Độ chính xác định vị điển hình của các công nghệ theo tài liệu

Công nghệ	Nguyên lý đo điển hình	Độ chính xác theo tài liệu	Hoạt động trong nhà
UWB	Thời gian bay (ToF / TWR)	$\approx 10 - 30$ cm [6], [7]	Có
BLE (Bluetooth Low Energy)	Cường độ tín hiệu (RSSI)	$\approx 1 - 3$ m [6]	Có
Wi-Fi RTT (IEEE 802.11mc/FTM)	Thời gian bay (Fine Timing Measurement)	$\approx 1 - 2$ m [6]	Có
Wi-Fi RSSI (fingerprinting)	Cường độ tín hiệu (RSSI)	$\approx 2 - 6$ m [6]	Có
GPS / GNSS	Thời gian truyền từ vệ tinh	$\approx 3 - 5$ m (ngoài trời); không khả dụng trong nhà [6]	Không

Ghi chú: các giá trị trong Bảng 4.1 là khoảng độ chính xác điển hình được báo cáo trong tài liệu khảo sát [6], [7], có thể thay đổi tùy điều kiện triển khai (số lượng và hình học điểm tham chiếu, mức độ che khuất NLOS, hiệu chuẩn). Bảng này bổ sung cho Bảng 2.1 ở Chương 2 (so sánh tổng quan GPS/Wi-Fi-BLE/UWB) bằng cách tách riêng từng công nghệ và gắn nguồn định lượng cụ thể.

Trên cơ sở các độ đo và bố trí nêu trên, kết quả định lượng sẽ được tổng hợp thành bảng và đồ thị so sánh giữa hai điều kiện truyền sóng cũng như giữa hai khâu xử lý (trước và sau bộ lọc). (Số liệu định lượng sẽ được bổ sung sau khi tiến hành đo đạc trên hệ thống thực tế.)

4.6 Phân tích và khai thác dữ liệu vận hành

Toàn bộ lịch sử vị trí phương tiện được lưu vào bảng chuỗi thời gian `vehicle\_history`, tạo nền tảng cho việc phân tích vận hành ngoài chức năng định vị thời gian thực. Hệ thống bổ sung một mô-đun phân tích (AnalyticsService) trên IoT Server, cung cấp ba nhóm chỉ tiêu được tính trực tiếp từ lịch sử vị trí và lộ ra qua REST API theo từng tầng:

- **Thời gian lưu bãi (dwell-time) theo phương tiện:** tổng thời gian mỗi xe hiện diện trên tầng, được tách thành các phiên đỗ — khoảng cách thời gian giữa hai bản ghi liên tiếp vượt ngưỡng (mặc định 5 phút) được coi là xe đã rời đi rồi quay lại, nhờ đó thời gian vắng mặt không bị tính vào thời gian đỗ.
- **Bản đồ nhiệt mức độ sử dụng (occupancy heatmap):** chia mặt bằng thành lưới ô (mặc định 0,5m), mỗi ô tích lũy tổng thời gian phương tiện lưu lại trong phạm vi ô đó; phần đóng góp của mỗi bước thời gian được giới hạn trần để một khoảng trống dữ liệu đơn lẻ không chi phối kết quả. Bản đồ nhiệt giúp nhận diện các khu vực "nóng" phục vụ tối ưu hóa bố trí ô đỗ.
- **Lưu lượng đỉnh (peak occupancy):** số phương tiện khác nhau hiện diện trong từng khung thời gian (mặc định 5 phút) cùng giá trị đỉnh toàn cục, phục vụ báo cáo công suất sử dụng bãi.

Các phép tính được tách thành hàm thuần (không phụ thuộc cơ sở dữ liệu) nên kiểm thử đơn vị được độc lập, bảo đảm tính đúng đắn của thuật toán tổng hợp. Kết

quả phân tích được giao diện web quản trị sử dụng để hiển thị báo cáo và biểu đồ, đồng thời là nguồn dữ liệu để chatbot trả lời các truy vấn mang tính thống kê như "khu vực nào hay được sử dụng nhất?" hoặc "xe X đã đỗ bao lâu?". Luồng xử lý từ lịch sử vị trí đến ba nhóm chỉ tiêu và phía tiêu thụ được mô tả trong Hình 4.6.

4.7 Đánh giá hợp đồng Tool → Component

Bên cạnh đường RAG, chatbot còn sinh thẻ giao diện (UI component) từ kết quả công cụ MCP. Khóa luận kiểm thử hợp đồng xác định (deterministic) giữa tầng công cụ (mcp-server) và tầng hiển thị (ứng dụng di động): nạp dữ liệu mẫu cố định vào hàm dựng component rồi kiểm tra schema, sự tồn tại renderer phía client, và khớp tham số.

Bảng 4.6: Kết quả kiểm thử hợp đồng Tool → Component (Lớp 1, xác định)

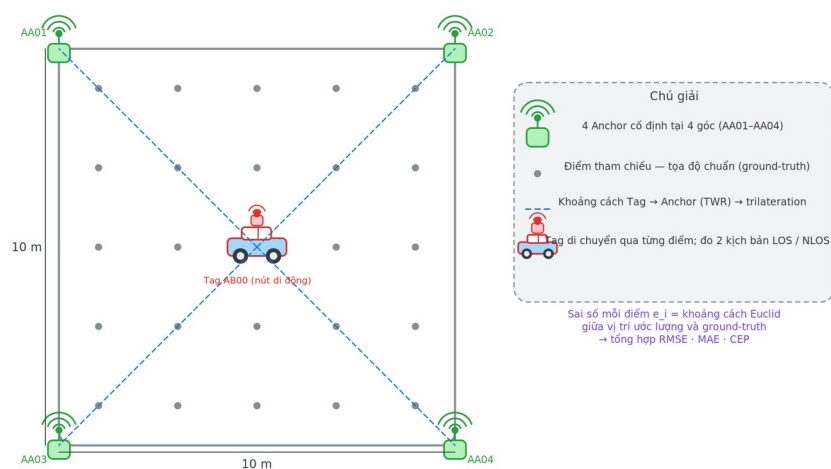
Độ đo	Giá trị
Schema-validity rate (props qua JSON Schema)	8/8 (100%)
% component thiếu renderer ở mobile	0% (0/8)
Argument-match (field bắt buộc đúng kiểu)	8/8 (100%)
Render-validity	8/8 (100%)
Catalog coverage (tool có test / tổng)	8/8 (100%)

Kết quả kiểm thử cho thấy cả 8 ánh xạ tool → component (7 component phân biệt; mobile có 12 loại renderer) đều hợp lệ về schema, có renderer tương ứng và khớp tham số — tầng trình bày không có lỗi hợp đồng nào. Quá trình kiểm thử còn phát hiện 5 component "mồ côi" (có renderer nhưng chưa có công cụ nào sinh ra: AnomalyAlertList, ParkingRecommendCard, VehicleEntryExitLog, VehicleInfoCard, VehicleOverviewListCard) — đây là bộ khung giao diện sẵn sàng cho tính năng tương lai mà không cần thay đổi phía mobile.

Kết luận chương 4. Chương 4 đã trình bày kết quả hiện thực hóa hệ thống: môi trường triển khai (Docker/VPS), các nhóm tính năng trên ứng dụng di động (bản

đồ 3D thời gian thực, quản lý xe, tìm xe qua bản đồ UWB và giải pháp đề xuất Tag cầm tay+BLE, khảo sát mặt bằng, phát hiện đồ sai, cảnh báo an ninh geofence/chống trộm, đề xuất bố trí tự động, chế độ Standard/Valet, thông báo), chatbot AI tiếng Việt (MCP+RAG đạt 100% trên n=101 câu, chấm khách quan) với vòng lặp agentic và dự phòng đa nhà cung cấp, giao diện web quản trị, phương pháp đánh giá định lượng độ chính xác định vị (RMSE/MAE/CEP/P95, kết quả mô phỏng §4.5.4) và mô-đun phân tích dữ liệu vận hành. Việc đo đạc định lượng trên hệ thống thực được tiến hành theo phương pháp đã chuẩn hóa ở Mục 4.5.

BỐ TRÍ THỰC NGHIỆM ĐÁNH GIÁ ĐỘ CHÍNH XÁC ĐỊNH VỊ UWB

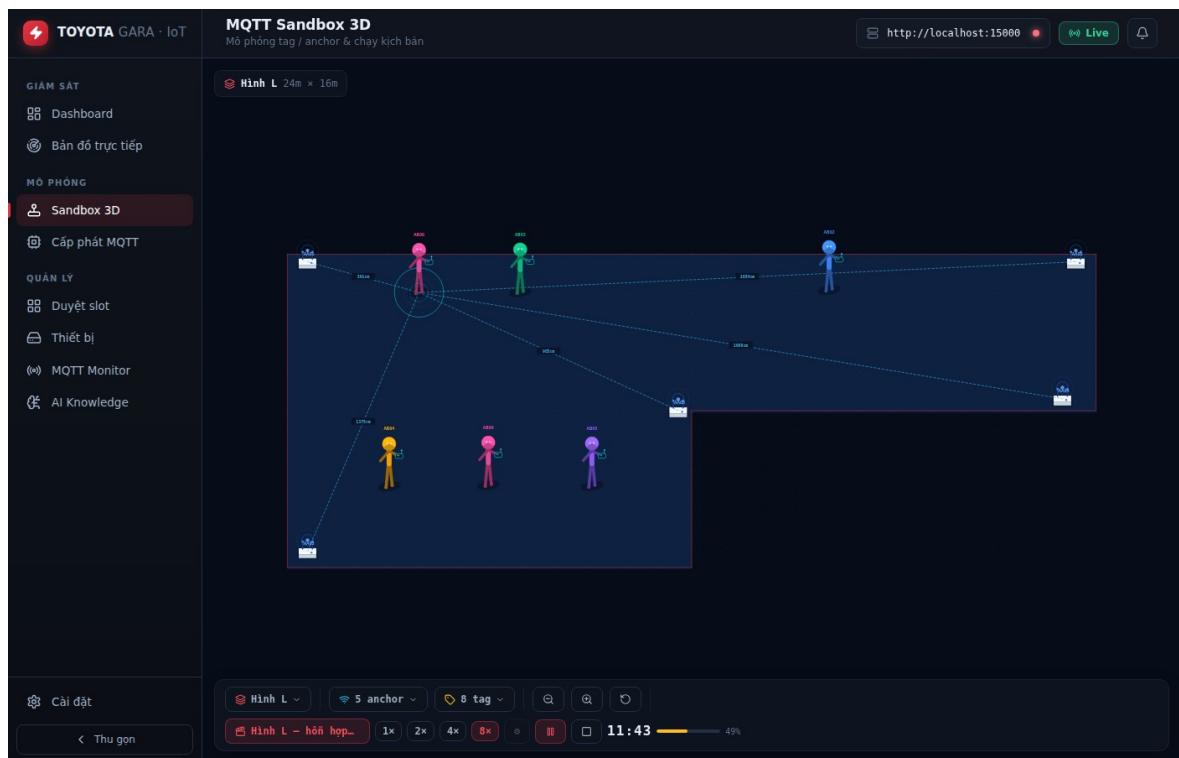


Hình 4.1: Bố trí thực nghiệm đánh giá định vị UWB (bãi 10m×10m, 4 Anchor tại 4 góc)

Hình 4.2: Giao diện bản đồ IoT 3D isometric — xe thật trên bản đồ

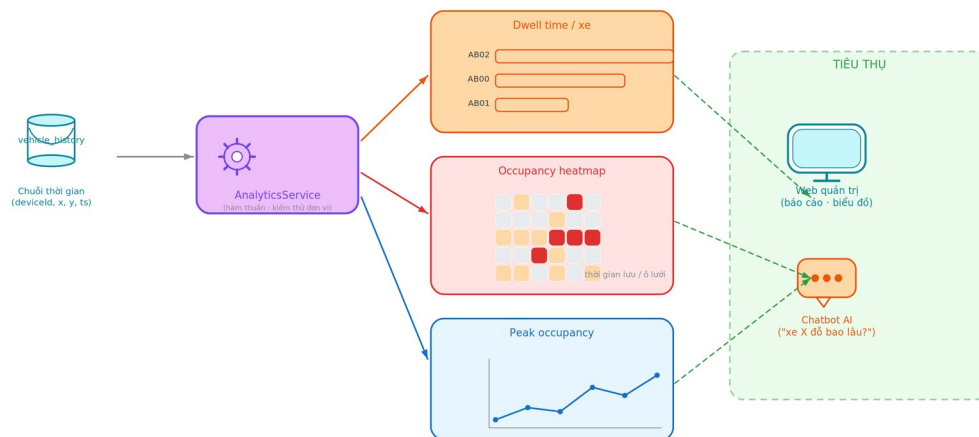
Hình 4.3: Màn hình chatbot AI — truy vấn vị trí xe bằng tiếng Việt

Hình 4.4: Giải pháp đề xuất tìm xe — người cầm Tag UWB di động dẫn hướng đến xe



Hình 4.5: MQTT Simulator Sandbox — kịch bản nhiều xe (mặt bằng hình L, các xe đã đến và đỗ vào ô lúc 11:43 giờ mô phỏng)

PHÂN TÍCH DỮ LIỆU VẬN HÀNH TỪ LỊCH SỬ VỊ TRÍ



Hình 4.6: Pipeline phân tích dữ liệu vận hành (dwell-time, heatmap, peak occupancy)

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Sau quá trình nghiên cứu và triển khai thực nghiệm, đề tài đã hoàn thành các mục tiêu đề ra:

1. Xây dựng và triển khai hoàn chỉnh hệ thống định vị UWB cho môi trường trong nhà; đã chuẩn hóa quy trình đánh giá định lượng độ chính xác (RMSE/MAE/CEP/P95) và triển khai công cụ tái lập tự động (xem Mục 4.5); kết quả đo thực địa đang tiến hành.
2. Triển khai kiến trúc AIoT hoàn chỉnh từ phần cứng đến ứng dụng di động và giao diện quản trị web.
3. Tích hợp thành công chatbot AI tiếng Việt với dữ liệu thời gian thực qua kỹ thuật Hybrid RAG. Bảng chứng chủ đạo khách quan (§4.3.8): trên bộ $n=101$ câu được chấm tự động bằng key-fact recall (không LLM-judge), cấu hình **MCP+RAG đạt 100%** ($G1=100\%$, $G2=100\%$, Wilson 95% CI: 96–100%); bổ sung: hệ thống truy hồi Hybrid đạt **Recall@5 = 0,821**, **nDCG@5 = 0,809**, **MRR = 0,962** trên bộ 13 câu gán nhãn thủ công (chi tiết §4.3.6, chỉ tiêu truy hồi offline).
4. Bổ sung mô-đun phân tích dữ liệu vận hành (thời gian lưu bãi, bản đồ nhiệt mức độ sử dụng và lưu lượng đỉnh) khai thác lịch sử vị trí để phục vụ báo cáo và truy vấn qua chatbot (xem Mục 4.6).

Hệ thống đã vận hành thông suốt trên môi trường thực, giải quyết bài toán quản lý bãi đỗ xe thông minh; việc kiểm chứng định lượng độ chính xác định vị đang được tiến hành theo phương pháp trình bày ở Mục 4.5.

5.2 Hạn chế của đề tài

Mặc dù đạt kết quả khả quan, hệ thống vẫn tồn tại một số hạn chế:

- Độ chính xác định vị vẫn bị ảnh hưởng bởi các vật cản kim loại lớn gây nhiễu NLOS mạnh.
- Chi phí phần cứng UWB hiện tại vẫn cao hơn so với BLE/Wi-Fi.

- Hệ thống mới chỉ thử nghiệm với số lượng Tag và Anchor hạn chế.
- Chưa hoàn tất đo đạc định lượng độ chính xác định vị trên hệ thống thực; các số liệu RMSE/MAE/CEP và độ trễ đầu–cuối sẽ được bổ sung theo phương pháp đã chuẩn hóa ở Mục 4.5.

5.3 Hướng phát triển tương lai

- Mở rộng hệ thống định vị 3D (x, y, z) để ứng dụng cho các bãi đỗ xe nhiều tầng.
- Tối ưu hóa thuật toán định vị khi số lượng phương tiện (Tag) tăng lớn (tối ưu hóa băng thông UWB).
- Tích hợp thêm các cảm biến môi trường (khói, nhiệt độ) để nâng cao tính an toàn cho garage.

CHƯƠNG 6: TÀI LIỆU THAM KHẢO

1. IEEE, "IEEE Std 802.15.4z-2020 — IEEE Standard for Low-Rate Wireless Networks, Amendment 1: Enhanced Ultra Wideband (UWB) Physical Layers (PHYs) and Associated Ranging Techniques," IEEE, Aug. 2020. doi: 10.1109/IEEESTD.2020.9179124.
2. G. Casiez, N. Roussel, and D. Vogel, "1€ Filter: A Simple Speed-based Low-pass Filter for Noisy Input in Interactive Systems," in Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI '12), Austin, TX, USA, 2012, pp. 2527–2530. doi: 10.1145/2207676.2208639.
3. P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. 34th Conf. Neural Information Processing Systems (NeurIPS), Vancouver, Canada, 2020. arXiv:2005.11401.
4. NestJS Documentation. [Online]. Available: <https://docs.nestjs.com>
5. Model Context Protocol (MCP) Specification. [Online]. Available: <https://modelcontextprotocol.io>
6. F. Zafari, A. Gkelias, and K. K. Leung, "A Survey of Indoor Localization Systems and Technologies," IEEE Communications Surveys & Tutorials, vol. 21, no. 4, pp. 2568–2599, 2019. doi: 10.1109/COMST.2019.2911558.
7. F. Che, Q. Z. Ahmed, P. I. Lazaridis, P. Sureephong, and T. Alade, "Indoor Positioning System (IPS) Using Ultra-Wide Bandwidth (UWB) — For Industrial Internet of Things (IIoT)," Sensors, vol. 23, no. 12, art. 5710, 2023. doi: 10.3390/s23125710.
8. OASIS, "MQTT Version 5.0," OASIS Standard, A. Banks, E. Briggs, K. Borgendale, and R. Gupta, Eds., 7 Mar. 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
9. G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in Proc. 32nd

- Int. ACM SIGIR Conf. Research and Development in Information Retrieval (SIGIR '09), Boston, MA, USA, 2009, pp. 758–759. doi: 10.1145/1571941.1572114.
10. S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. doi: 10.1561/15000000019.
 11. D. H. Douglas and T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature," *Cartographica*, vol. 10, no. 2, pp. 112–122, 1973. doi: 10.3138/FM57-6770-U75U-7727.
 12. R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960. doi: 10.1115/1.3662552.
 13. Espressif Systems, "ESP32-C3 Series Datasheet," Espressif Systems, 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-c3_datasheet_en.pdf
 14. Anthropic, "Introducing Contextual Retrieval," Anthropic, Sep. 2024. [Online]. Available: <https://www.anthropic.com/news/contextual-retrieval>

TÀI LIỆU THAM KHẢO